



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

LABORATORIO DE COMPUTACIÓN GRÁFICA E
INTERACCIÓN HUMANO COMPUTADORA

PRÁCTICA 03

MODELADO GEOMÉTRICO

Grupo de laboratorio: 06

Profesor: Dr. Sergio Teodoro Vite

Integrantes:

- Basilio Illescas Andrés
- Cruz Macedo Samuel Santiago
- Medel Piña Ángel de Jesús
- Pérez Olvera Alexis Abraham
- Segura Cedeño Luisa María



Fecha de entrega: Martes 22 de septiembre del 2026

Semestre: 2027-1

MODELADO GEOMÉTRICO

- 321137135 andres.basilio@ingenieria.unam.edu
- 424065465 santi.sam120@gmail.com
- 321258173 angelmedel130105@gmail.com
- 321150503 ralexisabrahamr@gmail.com
- 320224159 segura.cedeno.luisa@gmail.com

1. Resumen

Resumen

En esta práctica se abordó el modelado geométrico, es decir, la construcción de objetos tridimensionales a partir de primitivas más sencillas. Se implementaron con OpenGL 3.3 las tres primeras primitivas que solicita el manual: un plano paralelo al plano XY de 2×2 unidades, un cubo de $2 \times 2 \times 2$ unidades y un círculo de radio unitario, las tres centradas en el origen del sistema de referencia y descritas mediante listas de vértices e índices que la clase `Mesh` envía a la unidad de procesamiento gráfico. Cada figura puede observarse desde dos puntos de vista distintos, asignados a las teclas `F1` y `F2`, y puede alternarse entre relleno y alambre para exhibir la triangulación con la que fue construida. El trabajo dejó ver que una primitiva geométrica no se dibuja directamente: antes se descompone en primitivas gráficas, en este caso triángulos, y de la forma en que se elija esa descomposición dependen tanto el número de vértices como la apariencia final. Al ejecutar la clase `Circle` del proyecto base detectamos que dejaba sin inicializar el color de casi todos sus vértices y que su arreglo de índices rebasaba el tamaño del arreglo de vértices, por lo que se reescribió como un abanico de triángulos con un único vértice central.

Abstract

This laboratory session deals with geometric modelling, that is, the construction of three-dimensional objects out of simpler primitives. The first three primitives requested by the manual were implemented in OpenGL 3.3: a plane parallel to the XY plane measuring 2×2 units, a cube measuring $2 \times 2 \times 2$ units and a circle of unit radius, all three centred at the origin of the reference system and described by vertex and index lists that the `Mesh` class uploads to the graphics processing unit. Each figure can be observed from two different viewpoints, bound to the `F1` and `F2` keys, and can be switched between filled and wireframe rendering in order to expose the triangulation it was built from. The work made clear that a geometric primitive is not drawn directly: it is first decomposed into graphic primitives, triangles in this case, and both the vertex count and the final appearance depend on how that decomposition is chosen. When running the `Circle` class supplied with the base project we found that it left the colour of nearly all its vertices uninitialised and that its index array exceeded the size of the vertex array, so it was rewritten as a triangle fan with a single central vertex.

2. Introducción

En esta tercera práctica del Laboratorio de Computación Gráfica el trabajo se centró en el modelado geométrico. En la Práctica 02 habíamos construido una casa, pero el interés estaba puesto en la cámara y en las matrices que determinan cómo se le observa; aquí ocurre lo contrario: la cámara se deja casi fija y lo que se estudia es la figura misma, es decir, cómo se decide qué vértices la componen y de qué manera se unen entre sí. Un modelo geométrico es la representación de las características geométricas de una entidad, concreta o abstracta, y se construye a partir de primitivas, esto es, de elementos más sencillos que el objeto que se quiere representar [11].

Conviene distinguir dos nociones que en el lenguaje cotidiano se confunden. Una *primitiva geométrica* es la figura que el usuario tiene en mente: un plano, un cubo, un círculo, una esfera. Una *primitiva gráfica*, en cambio, es lo que la interfaz sabe dibujar realmente, que en el perfil *core* de OpenGL se reduce a puntos, líneas y triángulos [5]. Entre una y otra hay siempre un paso de descomposición: el cubo no se envía a la tarjeta como «un cubo», sino como doce triángulos, y el círculo, que en el papel es una curva continua, se aproxima con un polígono de n lados cuyo error disminuye conforme n crece. Esa aproximación es la razón por la que el modelado geométrico se estudia como un problema de parametrización y no sólo como una lista de coordenadas [3].

La estructura con la que se describen estas figuras es la malla poligonal, formada por un arreglo de vértices y un arreglo de índices que indica qué tercias de vértices forman cada triángulo [4]. Separar ambos arreglos permite que un mismo vértice participe en varias caras sin duplicarse, lo cual reduce la memoria empleada y, sobre todo, la cantidad de datos que se transfieren a la unidad de procesamiento gráfico. En el proyecto base esa estructura está encapsulada en la clase `Mesh`, que crea el objeto de

arreglo de vértices y los búferes correspondientes y dibuja el conjunto con `glDrawElements` [6]. Cada vértice lleva además un color expresado en el modo RGBA, en el que las componentes cromáticas y la de opacidad se manejan como valores flotantes acotados [2]; como el color se declara por vértice, la etapa de rasterización interpola los valores intermedios y produce los degradados que se aprecian en las capturas de este reporte.

Las tres actividades que se documentan aquí recorren esa idea en orden creciente de dificultad. El plano es el caso mínimo, con cuatro vértices y dos triángulos, y sirve para fijar la convención de que la figura se construye centrada en el origen. El cubo introduce la tercera dimensión y obliga a razonar por caras, comprobando que ninguna quede sin cubrir. El círculo introduce por fin la parametrización: sus vértices no se escriben a mano, sino que se calculan con las funciones trigonométricas evaluadas en incrementos regulares del ángulo, de modo que el número de lados se vuelve un parámetro del constructor. Para cada figura se asignaron dos vistas a las teclas **F1** y **F2**, tal como pide el manual, y se agregó una tecla que alterna el modo de rasterización entre relleno y alambre mediante `glPolygonMode`, con la que puede verse directamente la descomposición en triángulos [7].

El desarrollo se llevó a cabo en C++ sobre el proyecto base proporcionado por el profesor, que emplea OpenGL 3.3 con GLFW para la gestión de la ventana, GLAD para la carga de las funciones de la API y GLM para las operaciones con matrices y vectores. La compilación se realizó en Visual Studio siguiendo la configuración descrita en el manual de la práctica. Durante el desarrollo se utilizó además un modelo de lenguaje como apoyo para analizar el comportamiento de la clase `Circle` del proyecto base y para la revisión de redacción del documento [12].

3. Metodología de la práctica

3.1. Requerimientos de hardware y software

La práctica se desarrolló sobre el proyecto base proporcionado por el profesor, que se compila como una aplicación de escritorio de C++ enlazada contra OpenGL. En la Tabla 1 se resumen los recursos que se emplearon; todos ellos cumplen con lo indicado en el manual de la práctica, que pide Windows 10 o Linux y Visual Studio 2017 o superior [11].

Elemento	Especificación empleada
Sistema operativo	Windows 11 (64 bits)
Entorno de desarrollo	Microsoft Visual Studio, conjunto de herramientas MSVC, plataforma x64
Lenguaje	C++ (estándar c++20)
API gráfica	OpenGL 3.3, perfil <i>core</i>
Bibliotecas	GLFW (creación de la ventana y manejo de eventos), GLAD (carga de las funciones de OpenGL) y GLM (álgebra de vectores y matrices)
Hardware	Equipo de cómputo con adaptador gráfico, integrado o dedicado, compatible con OpenGL 3.3
Archivos de apoyo	<code>shaders/02-simplePVM.vs</code> y <code>shaders/02-simplePVM.fs</code> , ubicados en el directorio de trabajo del ejecutable

Tabla 1: Requerimientos de hardware y software utilizados en la práctica.

La configuración del proyecto siguió los mismos pasos de la práctica anterior: se creó una carpeta de proyecto dentro de `03_GeomModelling`, se agregó el archivo de origen, se fijó el directorio de salida y el de trabajo en `bin`, y se declararon los directorios de inclusión (`deps/glad`, `deps/glfw`, `deps/glm` y el directorio `include` de la práctica), los directorios de bibliotecas y las entradas del vinculador (`opengl32.lib`, `glad.lib` y `glfw3.lib`). El directorio de trabajo es indispensable porque los shaders

se cargan con una ruta relativa. Se reutilizaron los shaders `02-simplePVM.vs` y `02-simplePVM.fs` de la práctica anterior, ya que la cadena de transformaciones no cambia en esta práctica.

3.2. Arquitectura del programa y flujo de ejecución

El programa conserva la organización del proyecto base: la función `Start()` se ejecuta una sola vez y prepara la ventana, el contexto de OpenGL, los shaders y las geometrías, mientras que `Update()` se ejecuta una vez por cuadro y se encarga de leer el teclado, armar las matrices y dibujar. En la Figura 1 se muestra el flujo completo. Al igual que en la práctica anterior, la lectura del teclado ocurre *antes* de construir las matrices, pues de lo contrario la imagen desplegada correspondería siempre al estado del cuadro anterior.

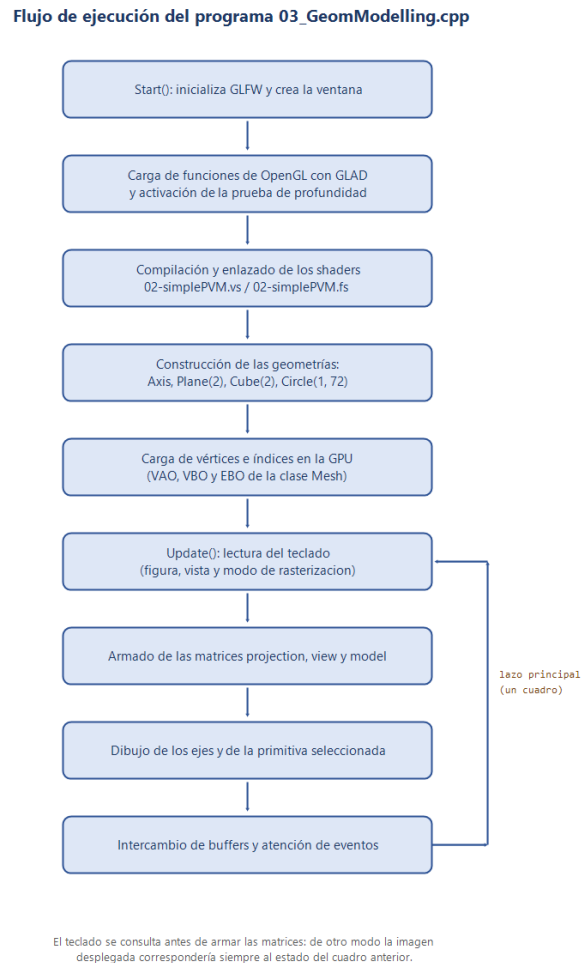


Figura 1: Diagrama de flujo del programa `03_GeomModelling.cpp`.

Sobre esa estructura se hicieron tres ajustes de carácter general. El primero fue interpretar el segundo vector de la cámara como el punto al que se mira y no como una dirección, que es lo que espera `glm::lookAt` [8]; el proyecto base lo inicializa con $(0, 0, -1)$, lo cual funciona por casualidad porque ese punto queda casi alineado con el origen, pero deja de funcionar en cuanto la cámara se coloca sobre otro eje. El segundo fue agregar `glfwSwapInterval(1)` después de crear el contexto, de modo que el lazo principal quede acotado a la frecuencia de actualización del monitor [9]. El tercero fue sustituir los seis bloques de dibujo que el proyecto base trae comentados, idénticos entre sí salvo por el nombre de la malla, por una sola función `dibujarMalla()` que recibe la malla y aplica la cadena proyección–vista–modelo; con ello agregar una primitiva se reduce a una línea en un `switch`. En el

Código 1 se muestra esa función junto con el cuerpo de Update().

```
1 // Dibuja una malla aplicando la cadena de transformaciones proyeccion-vista-modelo.
2 void dibujarMalla(Mesh* malla) {
3     if (malla == nullptr)
4         return;
5
6     glUseProgram(myShader->ID);
7
8     glm::mat4 projection = glm::perspective(glm::radians(45.0f),
9                                             (float)screenWidth / (float)screenHeight,
10                                            0.1f, 1000.0f);
11     myShader->setMat4("projection", projection);
12
13     glm::mat4 view = glm::lookAt(camera.Position, camera.View, camera.Up);
14     myShader->setMat4("view", view);
15
16     // Las primitivas ya se construyen centradas en el origen,
17     // así que la matriz de modelo es la identidad.
18     glm::mat4 model = glm::mat4(1.0f);
19     myShader->setMat4("model", model);
20
21     malla->Draw(*myShader);
22
23     glUseProgram(0);
24 }
25
26 bool Update() {
27     keyboardCallback(window);
28
29     glClearColor(bgR, bgG, bgB, bgA);
30     glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
31
32     // Los ejes se dibujan siempre en relleno para que sigan siendo visibles
33     // cuando la figura esta en modo alambre.
34     glPolygonMode(GL_FRONT_AND_BACK, GL_FILL);
35     dibujarMalla(meshAxis);
36
37     glPolygonMode(GL_FRONT_AND_BACK, modoAlambre ? GL_LINE : GL_FILL);
38
39     switch (figuraActual) {
40         case 1: dibujarMalla(meshPlane); break; // Actividad 1
41         case 2: dibujarMalla(meshCube); break; // Actividad 2
42         case 3: dibujarMalla(meshCircle); break; // Actividad 3
43         default: break;
44     }
45
46     glPolygonMode(GL_FRONT_AND_BACK, GL_FILL);
47
48     glfwSwapBuffers(window);
49     glfwPollEvents();
50
51     return true;
52 }
```

Listing 1: Funcion de dibujo comun a las tres primitivas y cuerpo de Update()

Las tres primitivas de esta entrega comparten un mismo principio de construcción, resumido en la Figura 2: la figura se define en su propio sistema de coordenadas y centrada en el origen, se descompone en triángulos y se entrega a la clase **Mesh** como un par de arreglos, uno de vértices y otro de índices. Lo único que cambia de una figura a otra es la regla con la que se generan esos arreglos.

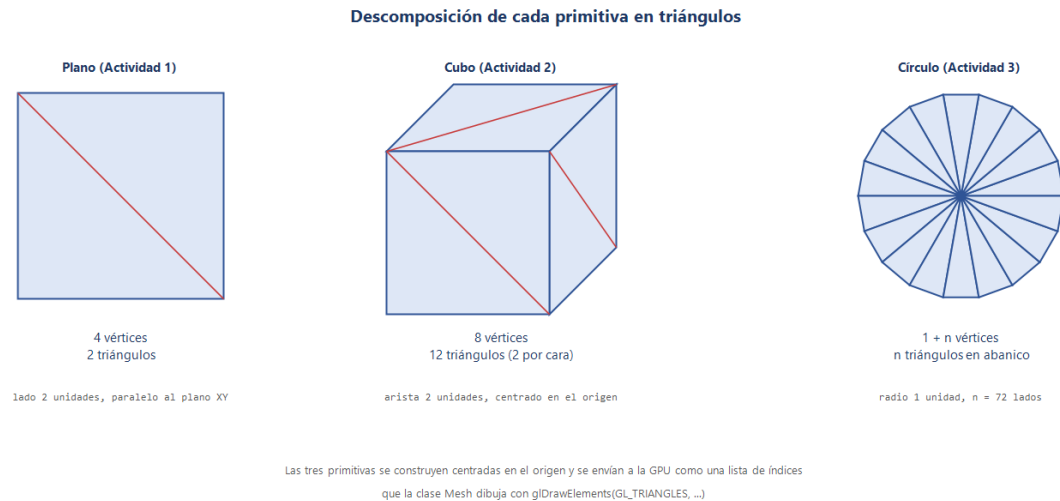


Figura 2: Descomposición de las tres primitivas en triángulos, con el número de vértices e índices que genera cada una.

3.3. Vistas y modo de rasterización

El manual pide que cada geometría pueda observarse desde dos perspectivas distintas mediante las teclas F1 y F2. Como en esta práctica la cámara no es el objeto de estudio, ambas vistas se resolvieron reasignando los tres vectores que recibe `glm::lookAt`: F1 coloca al observador sobre el eje $+Z$, a cinco unidades del origen, con lo que se obtiene la vista frontal, y F2 lo coloca en el punto $(3, 3, 3)$, sobre la diagonal del primer octante, con lo que se obtiene la vista oblicua. En ambos casos el punto observado es el origen y el vector de orientación es $(0, 1, 0)$.

Se agregó además la tecla L, que alterna el modo de rasterización entre relleno y alambre con `glPolygonMode` [7]. Esta tecla no la pide el manual, pero resultó indispensable para las tres actividades, porque es la única forma de comprobar a simple vista que la descomposición en triángulos es la que se pretendía: en modo relleno, un triángulo faltante y un triángulo repetido producen a menudo la misma imagen. El cambio de modo se detecta por flanco y no por estado, ya que el teclado se consulta una vez por cuadro y, de otro modo, una sola pulsación alternaría el modo decenas de veces. En el Código 2 se muestra el manejo del teclado completo.

```

1 void keyboardCallback(GLFWwindow* window) {
2
3     if (glfwGetKey(window, GLFW_KEY_ESCAPE) == GLFW_PRESS) {
4         glfwSetWindowShouldClose(window, true);
5     }
6
7     // Selección de la primitiva que se despliega
8     if (glfwGetKey(window, GLFW_KEY_1) == GLFW_PRESS) {
9         figuraActual = 1; // Actividad 1: plano
10    }
11    if (glfwGetKey(window, GLFW_KEY_2) == GLFW_PRESS) {
12        figuraActual = 2; // Actividad 2: cubo
13    }
14    if (glfwGetKey(window, GLFW_KEY_3) == GLFW_PRESS) {
15        figuraActual = 3; // Actividad 3: círculo
16    }
17
18    // Alternancia entre relleno y alambre. Se detecta el flanco de la tecla
19    // porque el teclado se consulta una vez por cuadro y, de lo contrario,
20    // el modo cambiaría decenas de veces con una sola pulsación.
21    bool teclaLActual = (glfwGetKey(window, GLFW_KEY_L) == GLFW_PRESS);

```

```

22     if (teclaActual && !teclaPrevia) {
23         modoAlambre = !modoAlambre;
24     }
25     teclaPrevia = teclaActual;
26
27     // Vistas solicitadas por la practica
28     if (glfwGetKey(window, GLFW_KEY_F1) == GLFW_PRESS) {
29         // Vista frontal: observador sobre el eje +Z mirando al origen
30         camera.Position = glm::vec3(0.0f, 0.0f, 5.0f);
31         camera.View      = glm::vec3(0.0f, 0.0f, 0.0f);
32         camera.Up        = glm::vec3(0.0f, 1.0f, 0.0f);
33     }
34     if (glfwGetKey(window, GLFW_KEY_F2) == GLFW_PRESS) {
35         // Vista oblicua: observador sobre la diagonal del primer octante
36         camera.Position = glm::vec3(3.0f, 3.0f, 3.0f);
37         camera.View      = glm::vec3(0.0f, 0.0f, 0.0f);
38         camera.Up        = glm::vec3(0.0f, 1.0f, 0.0f);
39     }
40 }

```

Listing 2: Selección de figura, vistas F1 y F2 y alternancia del modo de rasterización

3.4. Actividad 1: Plano paralelo al plano XY

La clase `Plane` del proyecto base ya construye un cuadrilátero contenido en el plano $z = 0$ con cuatro vértices y dos triángulos, de modo que la actividad se reduce a elegir correctamente el argumento del constructor. El punto que conviene subrayar es que ese constructor no recibe el semilado sino el lado completo: internamente calcula `value = size / 2.0f` y coloca los vértices en $(\pm value, \pm value, 0)$. Para obtener el plano de 2×2 unidades que pide el manual hay que invocarlo entonces con `Plane(2.0f)`, y no con `Plane(1.0f)` como aparece en el ejemplo comentado del proyecto base, que produce un plano de 1×1 . Esa división entre dos es también la que garantiza que la figura quede centrada en el origen para cualquier tamaño.

Los cuatro vértices llevan colores distintos (rojo, verde, azul y cian), de manera que la interpolación que realiza la etapa de rasterización produce un degradado continuo sobre las dos caras triangulares. En el Código 3 se reproduce la construcción de la clase y la línea con la que se instancia.

```

1 // --- Clase Plane del proyecto base (include/Plane.h) ---
2 void buildVertices(float size) {
3     float value = size / 2.0f; // el constructor recibe el lado completo
4
5     Vertex v;
6
7     v.Position.x = value; v.Position.y = value; v.Position.z = 0.0f;
8     v.Color.x = 1.0f; v.Color.y = 0.0f; v.Color.z = 0.0f;
9     vertices.push_back(v);
10
11     v.Position.x = -value; v.Position.y = value; v.Position.z = 0.0f;
12     v.Color.x = 0.0f; v.Color.y = 1.0f; v.Color.z = 0.0f;
13     vertices.push_back(v);
14
15     v.Position.x = -value; v.Position.y = -value; v.Position.z = 0.0f;
16     v.Color.x = 0.0f; v.Color.y = 0.0f; v.Color.z = 1.0f;
17     vertices.push_back(v);
18
19     v.Position.x = value; v.Position.y = -value; v.Position.z = 0.0f;
20     v.Color.x = 0.0f; v.Color.y = 1.0f; v.Color.z = 1.0f;
21     vertices.push_back(v);
22 }
23
24 void buildIndices() {
25     indices.push_back(0); indices.push_back(1); indices.push_back(2);

```

```

26     indices.push_back(0); indices.push_back(2); indices.push_back(3);
27 }
28
29 // --- Instanciacion en Start() ---
30 // Plano paralelo al plano XY, de 2x2 unidades, centrado en el origen.
31 Plane plane(2.0f);
32 meshPlane = new Mesh(plane.vertices, plane.indices);

```

Listing 3: Construccion del plano y su instanciacion con lado 2

3.5. Actividad 2: Cubo centrado en el origen

El cubo sigue el mismo criterio: la clase `Cube` recibe la arista completa y coloca los ocho vértices en $(\pm\text{value}, \pm\text{value}, \pm\text{value})$, por lo que el cubo de $2 \times 2 \times 2$ unidades se obtiene con `Cube(2.0f)`. Los primeros cuatro vértices corresponden a la cara $z = -1$ y los cuatro restantes a la cara $z = +1$, y las seis caras se arman con dos triángulos cada una, para un total de doce triángulos y treinta y seis índices.

Antes de darlo por bueno verificamos que la lista de índices efectivamente cubriera las seis caras, ya que un cubo al que le falta una cara se ve idéntico a uno completo mientras no se le gire lo suficiente. La comprobación se hizo agrupando los índices por caras: los vértices $\{0, 1, 2, 3\}$ forman la cara trasera, $\{4, 5, 6, 7\}$ la frontal, $\{0, 3, 4, 7\}$ la derecha, $\{1, 2, 5, 6\}$ la izquierda, $\{0, 1, 4, 5\}$ la superior y $\{2, 3, 6, 7\}$ la inferior; las seis aparecen en el arreglo. Conviene notar que el sentido de recorrido no es consistente entre todas ellas, lo cual no afecta el resultado porque en esta práctica no se activa el descarte de caras traseras, pero sí lo haría en prácticas posteriores. En el Código 4 se muestra la construcción de los índices y la instanciación.

```

1 // --- Clase Cube del proyecto base (include/Cube.h) ---
2 // Vertices 0..3: cara z = -value   Vertices 4..7: cara z = +value
3 void buildIndices() {
4     // Cara trasera (z = -value)
5     indices.push_back(0); indices.push_back(1); indices.push_back(2);
6     indices.push_back(0); indices.push_back(2); indices.push_back(3);
7
8     // Cara frontal (z = +value)
9     indices.push_back(4); indices.push_back(5); indices.push_back(6);
10    indices.push_back(4); indices.push_back(6); indices.push_back(7);
11
12    // Cara derecha (x = +value)
13    indices.push_back(0); indices.push_back(3); indices.push_back(4);
14    indices.push_back(3); indices.push_back(4); indices.push_back(7);
15
16    // Cara izquierda (x = -value)
17    indices.push_back(1); indices.push_back(5); indices.push_back(2);
18    indices.push_back(2); indices.push_back(5); indices.push_back(6);
19
20    // Cara superior (y = +value)
21    indices.push_back(0); indices.push_back(1); indices.push_back(5);
22    indices.push_back(0); indices.push_back(4); indices.push_back(5);
23
24    // Cara inferior (y = -value)
25    indices.push_back(2); indices.push_back(3); indices.push_back(6);
26    indices.push_back(3); indices.push_back(6); indices.push_back(7);
27 }
28
29 // --- Instanciacion en Start() ---
30 // Cubo de 2x2x2 unidades, centrado en el origen.
31 Cube cube(2.0f);
32 meshCube = new Mesh(cube.vertices, cube.indices);

```

Listing 4: Indices por cara del cubo e instanciacion con arista 2

3.6. Actividad 3: Círculo de radio unitario

El círculo es la primera figura de la práctica que no puede escribirse vértice por vértice, porque su contorno es una curva. Lo que se hace es aproximarlos con un polígono regular de n lados: se recorre el ángulo θ en incrementos de $360^\circ/n$ y en cada paso se evalúa

$$x = r \cos \theta, \quad y = r \sin \theta, \quad z = 0,$$

con lo que se obtienen los n vértices del contorno. La superficie se cubre después con un abanico de triángulos: un único vértice en el centro se une con cada par de vértices consecutivos del contorno, de modo que hacen falta n triángulos y $1 + n$ vértices en total.

La clase `Circle` del proyecto base sigue esta misma idea, pero al ejecutarla encontramos tres problemas que se documentan en la sección de experimentos: sólo asigna color a los vértices correspondientes a 0° , 120° y 240° , y deja los demás con el contenido indeterminado de la estructura `Vertex`; duplica el vértice del centro una vez por cada punto del contorno, de modo que emplea $2n$ vértices en lugar de $1 + n$; y su segundo lazo de índices llega a escribir el valor `vertices.size()`, que es un índice que no existe dentro del arreglo.

Por esa razón se reescribió la clase conservando su nombre y su interfaz, para no alterar el resto del programa. El archivo vive en la carpeta de código de la práctica, junto al archivo `.cpp`, y como el compilador resuelve `#include Circle.h` buscando primero en el directorio del archivo que lo incluye, esta versión tiene precedencia sobre la del directorio `include` sin necesidad de tocar el proyecto base. El constructor recibe ahora el radio y el número de lados, con valor predeterminado de 72, es decir, un vértice cada cinco grados. Para el color del contorno se conservaron los tres tonos del proyecto original (cian en 0° , magenta en 120° y azul en 240°), pero interpolados linealmente entre anclas, de manera que todos los vértices tengan un valor definido. En el Código 5 se muestra la clase completa.

```
1 class Circle {
2 public:
3     // radius: radio de la circunferencia.
4     // slices: numero de lados del poligono con el que se aproxima.
5     Circle(float radius, int slices = 72) {
6         if (slices < 3) slices = 3;
7         this->slices = slices;
8         buildVertices(radius);
9         buildIndices();
10    }
11
12    vector<Vertex>      vertices;
13    vector<unsigned int> indices;
14
15 private:
16     int slices;
17
18     // Se conservan los tres tonos del proyecto base (cian en 0 grados,
19     // magenta en 120 y azul en 240) pero ahora se interpolan para que
20     // todos los vertices tengan un valor definido.
21     vec4 colorContorno(float grados) {
22         const vec3 anclas[3] = {
23             vec3(0.0f, 1.0f, 1.0f),    // 0 grados
24             vec3(1.0f, 0.0f, 1.0f),    // 120 grados
25             vec3(0.0f, 0.0f, 1.0f)     // 240 grados
26         };
27
28         float t = grados / 120.0f;      // sector de 120 grados
29         int i = (int)t % 3;              // ancla inicial
30         float f = t - (float)((int)t);  // avance dentro del sector
31
32         vec3 c = anclas[i] * (1.0f - f) + anclas[(i + 1) % 3] * f;
33         return vec4(c, 1.0f);
34    }
```

```

35
36 void buildVertices(float radius) {
37     Vertex v;
38
39     // Vertice 0: centro del circulo
40     v.Position = vec3(0.0f, 0.0f, 0.0f);
41     v.Color     = vec4(0.20f, 0.20f, 0.25f, 1.0f);
42     vertices.push_back(v);
43
44     // Vertices 1..slices: contorno sobre el plano XY
45     float paso = 360.0f / (float)slices;
46     for (int i = 0; i < slices; i++) {
47         float grados = paso * (float)i;
48         float rad     = radians(grados);
49
50         v.Position = vec3(radius * cos(rad), radius * sin(rad), 0.0f);
51         v.Color     = colorContorno(grados);
52         vertices.push_back(v);
53     }
54 }
55
56 void buildIndices() {
57     // Un triangulo por cada lado del poligono. El operador modulo cierra
58     // el circulo uniendo el ultimo vertice del contorno con el primero.
59     for (int i = 0; i < slices; i++) {
60         indices.push_back(0);
61         indices.push_back(1 + i);
62         indices.push_back(1 + (i + 1) % slices);
63     }
64 }
65 };
66
67 // --- Instanciacion en Start() ---
68 // Circulo de radio 1 unidad, centrado en el origen, con 72 lados.
69 Circle circle(1.0f, 72);
70 meshCircle = new Mesh(circle.vertices, circle.indices);

```

Listing 5: Clase Circle reescrita como abanico de triangulos

3.7. Actividad 4: Cilindro centrado en el origen

Para esta actividad se construyó un cilindro tridimensional de radio unitario ($r = 1,0$) y altura de dos unidades ($h = 2,0$) centrado en el origen del sistema cartesiano. Al analizar la plantilla base del archivo `Cylinder.h`, se encontró que el cuerpo se dibujaba acostado a lo largo del eje Z , le faltaban las tapas circulares y la mayor parte de los vértices quedaban sin asignación de color.

La metodología consistió en reescribir la clase utilizando parametrización de radio, altura y subdivisiones angulares:

1. **Generación de anillos perimétricos:** Se generaron dos círculos de 72 vértices cada uno sobre planos paralelos al plano XZ : el anillo inferior ubicado en $y = -1,0$ y el superior en $y = +1,0$. Las coordenadas horizontales se obtuvieron mediante $x = r \cos(\theta)$ y $z = r \sin(\theta)$ con incrementos angulares regulares.
2. **Interpolación de color:** Se aplicó una función de mezcla de colores suave que recorre diferentes tonalidades de azul en función del ángulo θ .
3. **Construcción del manto lateral:** Se conectaron los vértices de ambos anillos formando dos triángulos por cada sector angular, asegurando el orden horario y antihorario para la orientación de caras.

4. **Cierre de tapas:** Se agregaron dos vértices centrales adicionales: uno en $(0, +1, 0, 0)$ para la tapa superior y otro en $(0, -1, 0, 0)$ para la inferior. Cada tapa se construyó mediante un abanico de triángulos conectando el centro con cada segmento del perímetro.

El código implementado en la clase `Cylinder.h` fue:

```
1 void buildGeometry(float radius, float height, int sectors) {
2     float halfH = height * 0.5f; // Rango de -1.0 a +1.0
3
4     // 1. Anillo inferior (y = -halfH)
5     for (int i = 0; i < sectors; ++i) {
6         float theta = glm::radians(i * (360.0f / (float)sectors));
7         Vertex v;
8         v.Position = glm::vec3(radius * cos(theta), -halfH, radius * sin(theta));
9         glm::vec3 col = interpolatorColor((float)i / (float)sectors);
10        v.Color = glm::vec4(col * 0.85f, 1.0f);
11        vertices.push_back(v);
12    }
13
14    // 2. Anillo superior (y = +halfH)
15    for (int i = 0; i < sectors; ++i) {
16        float theta = glm::radians(i * (360.0f / (float)sectors));
17        Vertex v;
18        v.Position = glm::vec3(radius * cos(theta), halfH, radius * sin(theta));
19        glm::vec3 col = interpolatorColor((float)i / (float)sectors);
20        v.Color = glm::vec4(col, 1.0f);
21        vertices.push_back(v);
22    }
23
24    // 3. Triangulación del manto lateral
25    for (int i = 0; i < sectors; ++i) {
26        int next = (i + 1) % sectors;
27        int bot1 = i, bot2 = next;
28        int top1 = sectors + i, top2 = sectors + next;
29
30        indices.push_back(bot1); indices.push_back(top1); indices.push_back(bot2);
31        indices.push_back(top1); indices.push_back(top2); indices.push_back(bot2);
32    }
33
34    // 4. Tapa superior (centro en y = +halfH)
35    unsigned int centerTopIdx = vertices.size();
36    Vertex vTop;
37    vTop.Position = glm::vec3(0.0f, halfH, 0.0f);
38    vTop.Color = glm::vec4(1.0f, 1.0f, 1.0f, 1.0f);
39    vertices.push_back(vTop);
40
41    for (int i = 0; i < sectors; ++i) {
42        int next = (i + 1) % sectors;
43        indices.push_back(centerTopIdx);
44        indices.push_back(sectors + i);
45        indices.push_back(sectors + next);
46    }
47
48    // 5. Tapa inferior (centro en y = -halfH)
49    unsigned int centerBotIdx = vertices.size();
50    Vertex vBot;
51    vBot.Position = glm::vec3(0.0f, -halfH, 0.0f);
52    vBot.Color = glm::vec4(0.1f, 0.1f, 0.2f, 1.0f);
53    vertices.push_back(vBot);
54
55    for (int i = 0; i < sectors; ++i) {
56        int next = (i + 1) % sectors;
57        indices.push_back(centerBotIdx);
58        indices.push_back(next);
```

```

59     indices.push_back(i);
60 }
61 }

```

Listing 6: Construcción geométrica y tapas del cilindro en Cylinder.h.

En el archivo principal 03_GeomModelling.cpp, el objeto se instancia con las dimensiones pedidas y se dibuja aplicando una matriz de modelo unitaria en el origen:

```

1 // Inicializacion en Start()
2 Cylinder cylinder(1.0f, 2.0f, 72);
3 meshCylinder = new Mesh(cylinder.vertices, cylinder.indices);
4
5 // Renderizado en Update()
6 glm::mat4 model = glm::mat4(1.0f);
7 myShader->setMat4("model", model);
8
9 glPolygonMode(GL_FRONT_AND_BACK, modoAlambre ? GL_LINE : GL_FILL);
10 if (meshCylinder != nullptr) {
11     meshCylinder->Draw(*myShader);
12 }

```

Listing 7: Instanciación y dibujado del cilindro en 03_GeomModelling.cpp.

3.8. Actividad 5. Esfera centrada en el origen

La clase **Sphere** del proyecto base ya resuelve la mitad del problema: **buildVertices** recorre dos ángulos, la colatitud φ y el azimut θ , en incrementos de 5° , y para cada pareja evalúa

$$x = r \sin \varphi \cos \theta, \quad y = r \sin \varphi \sin \theta, \quad z = r \cos \varphi,$$

con $\varphi \in [0^\circ, 175^\circ]$ y $\theta \in [0^\circ, 355^\circ]$, lo que produce una rejilla de 36×72 vértices, 2592 en total. Lo que el proyecto base deja vacío es **buildIndices**, es decir, el método que debía triangular esa rejilla.

Como la rejilla que entrega **buildVertices** se guarda en un solo arreglo, recorriendo primero todos los valores de θ de una fila de φ antes de pasar a la siguiente, el vértice de la fila i (el índice de φ) y columna j (el índice de θ) vive en la posición $i \cdot \text{slices} + j$, con $\text{slices} = 72$. Con esa fórmula, cualquier celda de la rejilla, delimitada por dos filas y dos columnas consecutivas, queda definida por cuatro índices que se pueden calcular directamente sin recorrer nada. Cada celda se dividió en dos triángulos, como en el cubo y como se resume en la Figura 2, y la columna se cierra con el operador módulo para que la última, en $\theta = 355^\circ$, se conecte con la primera, en $\theta = 0^\circ$, sin dejar una costura abierta. El recorrido de filas se detiene en la penúltima, porque no existe una fila 36 con la que emparejarla. En el Código 8 se muestra el método completo.

```

1 void buildIndices() {
2     int stacks = 180 / 5; // filas de la rejilla (segun phi): 36
3     int slices = 360 / 5; // columnas de la rejilla (segun theta): 72
4
5     for (int i = 0; i < stacks - 1; i++) {
6         for (int j = 0; j < slices; j++) {
7             int actual = i * slices + j;
8             int siguiente = i * slices + (j + 1) % slices;
9             int abajo = (i + 1) * slices + j;
10            int abajoSiguiente = (i + 1) * slices + (j + 1) % slices;
11
12            indices.push_back(actual);    indices.push_back(siguiente);
13            indices.push_back(abajo);
14            indices.push_back(siguiente);  indices.push_back(abajoSiguiente);
15            indices.push_back(abajo);
16        }
17    }
18 }

```

Listing 8: Triangulación de la rejilla de la esfera por celdas

El segundo ajuste fue de color. Igual que en el círculo original, la clase sólo pinta los vértices cuyo θ vale 0° , 120° o 240° y deja el resto sin inicializar; a diferencia del círculo, aquí la condición no depende de φ , así que son 108 de los 2592 vértices (los tres meridianos completos) los que reciben color, y el resto queda indeterminado. En vez de repetir la interpolación por anclas que se usó en el círculo, se optó por un esquema más simple y habitual en gráficos por computadora para depurar geometría esférica: colorear cada vértice según su propia posición normalizada, es decir, tratar la dirección $(x, y, z)/r$ como si fuera directamente un color (r, g, b) , remapeando cada componente de $[-1, 1]$ a $[0, 1]$:

$$Color.r = 0,5 + 0,5 \frac{x}{r}, \quad Color.g = 0,5 + 0,5 \frac{y}{r}, \quad Color.b = 0,5 + 0,5 \frac{z}{r}.$$

Con esto los 2592 vértices quedan con un color definido y, como puntos cercanos en la esfera tienen posiciones normalizadas cercanas, la interpolación entre vértices vecinos produce una transición continua en toda la superficie.

3.9. Actividad 6: Cono centrado en el origen

Para esta actividad se construyó un cono de radio unitario ($r = 1,0$) y altura de dos unidades ($h = 2,0$), centrado en el origen del sistema de referencia. Al igual que en las primitivas anteriores, el cono no se dibuja como una superficie continua, sino que se descompone en un conjunto de triángulos procesados por la clase `Mesh`. Su geometría se resolvió mediante dos abanicos triangulares.

Al revisar la clase `Cone` que venía en el proyecto base (`include/Cone.h`), encontramos inconsistencias estructurales:

1. **Duplicación de vértices:** En `buildVertices` se recorre nuevamente la variable angular para colocar repetidamente el vértice del ápice en $(0, 0, size)$, generando n copias idénticas en lugar de un único punto compartido.
2. **Color no inicializado:** Únicamente se asigna color a los pasos angulares correspondientes a 0° , 120° y 240° , dejando el resto de los vértices con memoria residual no inicializada.
3. **Triángulos degenerados y falta de base:** El método `buildIndices` original genera caras sin área y omite la construcción de la base circular.

Por estas razones, volvimos a escribir la clase desde cero. Le pusimos como datos ajustables el radio de la base, la altura total y el número de lados del círculo, dejándolo en 72 para dar un paso de cinco grados en cada vuelta, igual que en las figuras anteriores.

Los puntos de la figura se programaron dentro de `buildVertices`. Como el cono debe medir 2 unidades de alto y quedar a la mitad del origen, dividimos la altura entre dos ($2,0/2,0 = 1,0$). Así, pusimos la punta de arriba (el ápice) en $z = 1,0$ y la base redonda en $z = -1,0$. La orilla de la base se formó girando en círculo con las funciones de seno y coseno:

$$x = r \cos \theta, \quad y = r \sin \theta, \quad z = -1,0 \tag{1}$$

lo que nos da los 72 puntos de la orilla. Después, agregamos un punto más en el centro de la base $(0, 0, -1,0)$ para poder taparlo por abajo. Para que no quedaran partes en negro o con fallas, pintamos el contorno combinando suavemente los colores cian, magenta y azul.

Para armar los triángulos en `buildIndices`, se hicieron dos recorridos que forman 144 caras en total:

- **Cuerpo del cono:** Se conectó la punta de arriba con cada parejita de puntos vecinos de la orilla, formando una especie de rebanadas que cierran los lados.
- **Tapa de abajo:** Se unió el punto del centro de la base con esos mismos puntos de la orilla para que el cono no quedara hueco por debajo.

En el Código 9 se presenta la implementación completa de la clase cone.h.

```
1 class Cone {
2 public:
3     Cone(float radius = 1.0f, float height = 2.0f, int slices = 72) {
4         if (slices < 3) slices = 3;
5         this->slices = slices;
6         buildVertices(radius, height);
7         buildIndices();
8     }
9
10    vector<Vertex>      vertices;
11    vector<unsigned int> indices;
12
13 private:
14     int slices;
15
16     vec4 colorContorno(float grados) {
17         const vec3 anclas[3] = {
18             vec3(0.0f, 1.0f, 1.0f), // 0 grados: cian
19             vec3(1.0f, 0.0f, 1.0f), // 120 grados: magenta
20             vec3(0.0f, 0.0f, 1.0f)  // 240 grados: azul
21         };
22         float t = grados / 120.0f;
23         int i = (int)t % 3;
24         float f = t - (float)((int)t);
25         vec3 c = anclas[i] * (1.0f - f) + anclas[(i + 1) % 3] * f;
26         return vec4(c, 1.0f);
27     }
28
29     void buildVertices(float radius, float height) {
30         Vertex v;
31         float halfH = height / 2.0f;
32
33         // Vertice 0: apice superior
34         v.Position = vec3(0.0f, 0.0f, halfH);
35         v.Color = vec4(0.2f, 0.2f, 0.8f, 1.0f);
36         vertices.push_back(v);
37
38         // Vertices 1..slices: perimetro de la base
39         float paso = 360.0f / (float)slices;
40         for (int i = 0; i < slices; i++) {
41             float rad = radians(paso * (float)i);
42             v.Position = vec3(radius * cos(rad), radius * sin(rad), -halfH);
43             v.Color = colorContorno(paso * (float)i);
44             vertices.push_back(v);
45         }
46
47         // Vertice slices + 1: centro de la base circular
48         v.Position = vec3(0.0f, 0.0f, -halfH);
49         v.Color = vec4(0.1f, 0.1f, 0.2f, 1.0f);
50         vertices.push_back(v);
51     }
52
53     void buildIndices() {
54         unsigned int apex = 0;
55         unsigned int centerBase = slices + 1;
56
57         for (int i = 0; i < slices; i++) {
58             unsigned int actual = 1 + i;
59             unsigned int siguiente = 1 + ((i + 1) % slices);
60
61             // Triangulos del manto lateral
62             indices.push_back(apex);
```

```

63         indices.push_back(actual);
64         indices.push_back(siguiete);
65
66         // Triangulos de la tapa inferior
67         indices.push_back(centerBase);
68         indices.push_back(siguiete);
69         indices.push_back(actual);
70     }
71 }
72 };
73
74 // Cono de radio 1 y altura 2 centrado en el origen (72 lados)
75 Cone cone(1.0f, 2.0f, 72);
76 meshCone = new Mesh(cone.vertices, cone.indices);

```

Listing 9: Clase Cone reescrita con centrado paramétrico y base cerrada

En el ciclo principal dentro de `Update()`, la geometría se integró al *switch* de selección en la tecla 6 mediante la función unificada `dibujarMalla()`. Debido a que la parametrización matemática alinea el eje del cono con el eje coordenado Z , se incluyó una rotación de -90° sobre el eje X en su matriz de modelo para orientar el ápice hacia el semieje $+Y$, posando la base de manera natural sobre el plano horizontal XZ [cite: 8, 9]. Finalmente, la alternancia con las teclas F1 y F2 permitió validar la perspectiva cónica, y la tecla L habilitó la inspección en modo alambre para corroborar que los 144 triángulos coincidieran con la estructura proyectada [?]

3.10. Actividad 7: Escena con OpenGL

La escena se armó dentro de la función `Update` reutilizando las geometrías construidas en las actividades anteriores. Cada objeto se dibuja con su propia matriz de modelo, que en el código se forma aplicando `glm::translate`, `glm::rotate` y `glm::scale` en ese orden. Como GLM multiplica por la derecha, la matriz resultante es

$$M = T R S,$$

por lo que sobre cada vértice se aplica primero el escalamiento, después la rotación y al final la traslación. Esto permitió escalar cada figura en sus propios ejes locales antes de orientarla y colocarla en la escena.

El primer elemento fue la superficie. El plano se escaló por un factor de 5 en sus tres ejes y se rotó 90° respecto al eje x , con lo que pasó de estar paralelo al plano XY a quedar acostado sobre el plano XZ , funcionando como el piso de la escena. No fue necesario trasladarlo, ya que se dejó centrado en el origen, como se muestra en el Código 10.

```

1 glm::mat4 model = glm::mat4(1.0f);
2 model = glm::translate(model, glm::vec3(0.0f, 0.0f, 0.0f));
3 model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
4 model = glm::scale(model, glm::vec3(5.0f, 5.0f, 5.0f));
5 myOtherShader->setMat4("model", model);
6
7 meshPlane->Draw(*myOtherShader);

```

Listing 10: Matriz de modelo de la superficie

Después se colocaron los cilindros. La clase `Cylinder` construye la figura a lo largo del eje z , entre $z = -1$ y $z = 1$, y el bloque de dibujo del código original ya incluía una rotación de 90° respecto al eje x que la deja de pie sobre el piso. Al cilindro grande se le redujo el largo escalándolo por 0.5 en el eje z y se trasladó a $(1,25, 0,5, -1,25)$; con la mitad de altura igual a 0.5 y el centro en $y = 0,5$, su base queda justo sobre la superficie. Los cilindros que funcionan como troncos de los árboles se escalaron en los tres ejes con $(0,1, 0,1, 0,6)$, lo que los hizo delgados y alargados, y se trasladaron a $(-1, 0,6, -1,25)$, $(-1, 0,6, 0,5)$, $(-2, 0,6, 0,5)$ y $(-1,5, 0,6, 2)$. En ellos no hizo falta modificar la rotación, ya que se conservó la misma de 90° que traía el código original, y al estar centrados en

$y = 0,6$ con esa misma mitad de altura, los troncos quedan apoyados en el piso. En el Código 11 se muestran las transformaciones del cilindro grande y del primer tronco; los otros tres troncos sólo cambian en los valores de la traslación.

```

1 // Cilindro grande
2 glm::mat4 model = glm::mat4(1.0f);
3 model = glm::translate(model, glm::vec3(1.25f, 0.5f, -1.25f));
4 model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
5 model = glm::scale(model, glm::vec3(1.0f, 1.0f, 0.5f));
6 myOtherShader->setMat4("model", model);
7
8 meshCylinder->Draw(*myOtherShader);
9
10 // Troncos
11 model = glm::mat4(1.0f);
12 model = glm::translate(model, glm::vec3(-1.0f, 0.6f, -1.25f));
13 model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
14 model = glm::scale(model, glm::vec3(0.1f, 0.1f, 0.6f));
15 myOtherShader->setMat4("model", model);
16
17 meshTronco->Draw(*myOtherShader);

```

Listing 11: Transformaciones del cilindro grande y del primer tronco

Posteriormente se agregaron las esferas, ya con la construcción de sus caras implementada en la Actividad 5. Las tres se escalonaron uniformemente por 0.25 en los tres ejes, lo que redujo su radio de 1 a 0.25, se rotaron 90° respecto al eje x y se trasladaron a $(2, 0,25, 2)$, $(1,25, 0,25, 2)$ y $(0,5, 0,25, 2)$. De esta forma quedan alineadas y descansando sobre la superficie, pues su radio coincide con la altura de su centro. El Código 12 muestra las transformaciones de la primera esfera.

```

1 glm::mat4 model = glm::mat4(1.0f);
2 model = glm::translate(model, glm::vec3(2.0f, 0.25f, 2.0f));
3 model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
4 model = glm::scale(model, glm::vec3(0.25f, 0.25f, 0.25f));
5 myOtherShader->setMat4("model", model);
6
7 meshSphere->Draw(*myOtherShader);

```

Listing 12: Transformaciones de la primera esfera

Por último se colocaron los conos. Al cono grande se le aplicó un escalamiento de $(1,2, 1,2, 0,5)$, que lo ensancha un poco y lo recorta en el eje z , y se trasladó a $(1,25, 1,5, -1,25)$, encima del cilindro grande, a manera de techo. Los conos pequeños, que funcionan como el follaje de los árboles, se escalonaron por 0.5 en los tres ejes y se trasladaron a las mismas coordenadas x y z de cada tronco, con $y = 1,7$, para que quedaran sobre ellos. En todos los conos no hizo falta modificar la rotación, ya que se conservó la de -90° respecto al eje x que traía el código original y que deja la punta hacia arriba. En el Código 13 se muestran las transformaciones del cono grande y de las hojas del primer árbol.

```

1 // Cono grande (techo)
2 glm::mat4 model = glm::mat4(1.0f);
3 model = glm::translate(model, glm::vec3(1.25f, 1.5f, -1.25f));
4 model = glm::rotate(model, glm::radians(-90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
5 model = glm::scale(model, glm::vec3(1.2f, 1.2f, 0.5f));
6 myOtherShader->setMat4("model", model);
7
8 meshCone->Draw(*myOtherShader);
9
10 // Arboles
11 model = glm::mat4(1.0f);
12 model = glm::translate(model, glm::vec3(-1.0f, 1.7f, -1.25f));
13 model = glm::rotate(model, glm::radians(-90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
14 model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
15 myOtherShader->setMat4("model", model);

```

```

16
17 meshArbol->Draw(*myOtherShader);

```

Listing 13: Transformaciones del cono grande y de las hojas del primer arbol

Con la geometría ya colocada, el siguiente paso fue asignar un color distinto a cada tipo de objeto. En las clases del proyecto base el color se guarda en cada vértice y sólo se asigna a los que tienen θ igual a 0° , 120° o 240° , así que todas las figuras del mismo tipo se veían iguales. Como el color forma parte de los datos que se envían al *buffer* cuando se crea el objeto **Mesh**, no basta con dibujar la misma malla varias veces: cada color necesita su propia malla. Por ello, en la función **Start** se construyó una figura por cada color, se recorrieron todos sus vértices para asignarles el mismo valor RGB y hasta entonces se creó su **Mesh**, como se muestra en el Código 14 para el cilindro grande.

```

1 Cylinder cylinder(1.0f);
2 for (auto& v : cylinder.vertices) {
3     v.Color.r = 0.80f; v.Color.g = 0.40f; v.Color.b = 0.20f; v.Color.a = 1.0f;
4 }
5 meshCylinder = new Mesh(cylinder.vertices, cylinder.indices);

```

Listing 14: Asignacion de color a todos los vertices del cilindro grande

El mismo procedimiento se repitió con el resto de las figuras, agregando las variables globales **meshTronco**, **meshSphere2**, **meshSphere3** y **meshArbol**, como se muestra en el Código 15. En **Update** únicamente se cambió qué malla se dibuja en cada llamada a **Draw**, sin modificar ninguna transformación; por ejemplo, los troncos pasaron de dibujarse con **meshCylinder** a dibujarse con **meshTronco**, y las hojas de los árboles con **meshArbol** en lugar de **meshCone**.

```

1 Mesh *mesh, *meshAxis, *meshPlane, *meshCircle,
2     *meshCylinder, *meshTronco, *meshCube, *meshSphere,
3     *meshSphere2, *meshSphere3,
4     *meshCone, *meshArbol;

```

Listing 15: Declaracion de las mallas de la escena

La superficie se dejó con sus colores originales. Los colores utilizados se resumen en la Tabla 2.

Objeto	Color	(<i>r</i> , <i>g</i> , <i>b</i>)
Cilindro grande	Naranja barro	(0,80, 0,40, 0,20)
Troncos	Café claro	(0,55, 0,40, 0,27)
Esfera 1	Rojo	(0,90, 0,10, 0,10)
Esfera 2	Amarillo	(1,00, 0,85, 0,10)
Esfera 3	Morado	(0,55, 0,20, 0,80)
Cono grande	Beige	(0,96, 0,87, 0,70)
Conos pequeños	Verde claro	(0,40, 0,75, 0,20)

Tabla 2: Colores asignados a cada objeto de la escena

4. Experimentos

4.1. Actividad 1: Plano paralelo al plano XY

La primera prueba se hizo descomentando el bloque del plano tal como viene en el proyecto base, es decir, con **Plane plane(1.0f)**. El plano apareció en pantalla y quedó centrado en el origen, pero era visiblemente más chico de lo que pedía el manual: sus vértices caen en $\pm 0,5$, de modo que el lado mide una unidad y no dos. En la Figura 3 se comparan las dos llamadas con la cámara en la misma posición, y se aprecia que el plano correcto ocupa exactamente el doble de lado que el del proyecto base. La confusión viene de que el parámetro parece ser el semilado, cuando en realidad el constructor

lo divide entre dos internamente; sólo al ver que las aristas del plano no coincidían con las marcas de los ejes coordenados nos dimos cuenta del error.

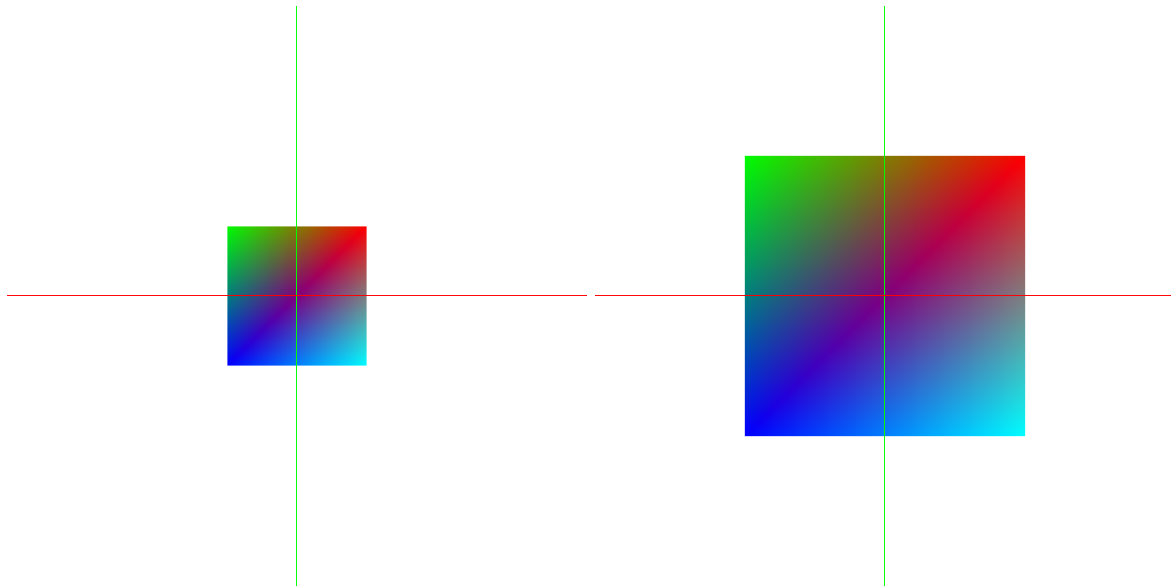


Figura 3: Comparación del tamaño del plano con la cámara fija en la vista frontal: a la izquierda, `Plane(1.0f)`, el valor que trae comentado el proyecto base, que produce un plano de 1×1 ; a la derecha, `Plane(2.0f)`, que produce el plano de 2×2 unidades que pide el manual.

Con el tamaño ya correcto probamos las dos vistas. Con **F1** el plano se ve como un cuadrado perfecto, porque la cámara está sobre el eje $+Z$ y el plano es paralelo al plano XY ; con **F2** la misma figura se ve como un paralelogramo, y es ahí donde se confirma que lo que hay en la escena es un objeto plano y no un cuadrado dibujado sobre la ventana. Ambas vistas aparecen en la Figura 4.

Un detalle inesperado apareció en la vista frontal: las líneas de los ejes X y Y se siguen viendo *encima* del plano, con su color íntegro, en lugar de quedar tapadas por él. Al principio pensamos que era una transparencia mal configurada. La explicación resultó más simple: los ejes y el plano están los dos contenidos en $z = 0$, es decir, exactamente a la misma profundidad, y como los ejes se dibujan primero y la prueba de profundidad usa `GL_LESS`, los fragmentos del plano no pasan la prueba en esos píxeles. Lo comprobamos midiendo el color de los píxeles sobre la línea, que resultó ser rojo puro $(255, 0, 0)$ y verde puro $(0, 255, 0)$, sin mezclarse con el degradado del plano.

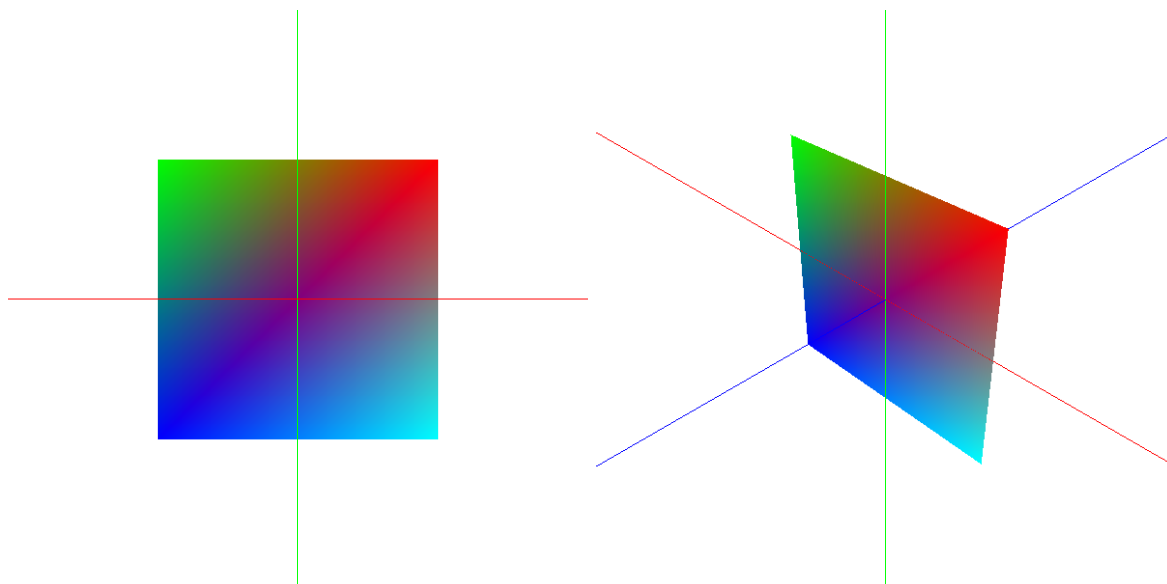


Figura 4: Las dos perspectivas del plano de 2×2 : vista frontal con F1 (izquierda) y vista oblicua con F2 (derecha).

Por último activamos el modo de alambre con la tecla L para verificar la descomposición. La Figura 5 muestra las dos aristas del contorno y la diagonal que separa los dos triángulos, que es exactamente lo que declara el arreglo de índices.

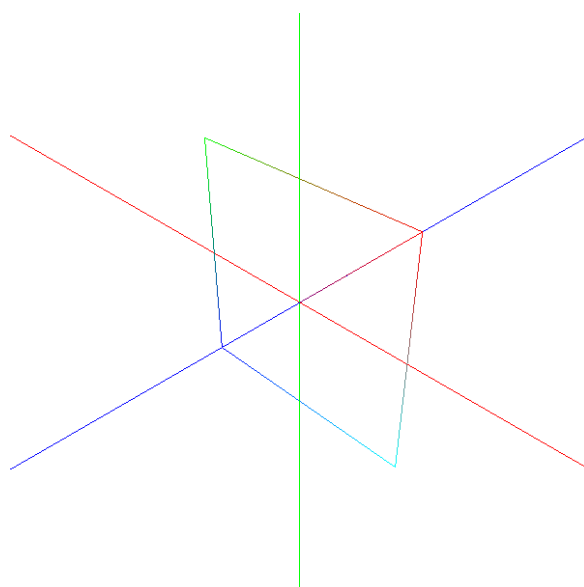


Figura 5: El plano en modo de alambre: se distinguen los cuatro vértices del contorno y la diagonal que divide la figura en dos triángulos.

4.2. Actividad 2: Cubo centrado en el origen

El cubo repitió la misma historia del tamaño: con `Cube(1.0f)`, el valor del proyecto base, la arista mide una unidad en lugar de dos. Como ya sabíamos qué buscar, esta vez la corrección fue inmediata; la comparación se muestra en la Figura 6. Aprovechamos además la vista oblicua para comprobar que la figura sí está centrada: los tres ejes coordenados entran por tres aristas distintas y salen por las opuestas, lo que sólo ocurre si el origen coincide con el centro geométrico del cubo.

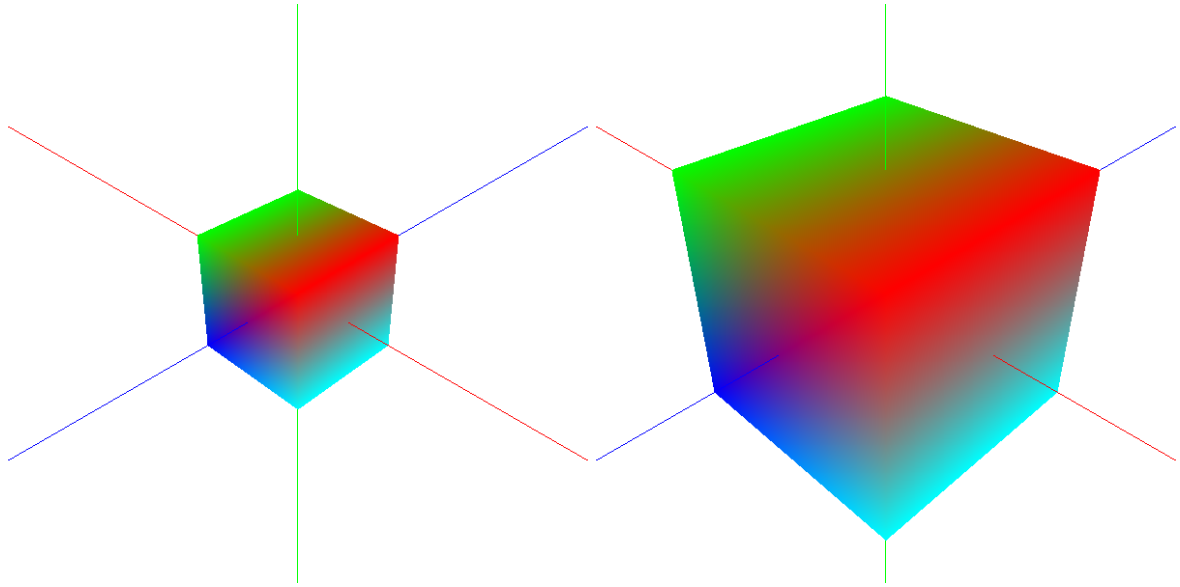


Figura 6: Efecto del argumento del constructor sobre el cubo, con la cámara fija en la vista oblicua: `Cube(1.0f)` a la izquierda y `Cube(2.0f)` a la derecha.

La prueba que más nos interesaba era la de las caras. En la vista frontal (Figura 7, izquierda) el cubo se ve como un cuadrado y resulta imposible saber si tiene seis caras o sólo una: la silueta sería idéntica en ambos casos. Al pasar a la vista oblicua aparecen tres caras al mismo tiempo y la figura se reconoce como un volumen, pero incluso así hay tres caras que no se ven nunca. Por eso recurrimos al modo de alambre, que se muestra en la Figura 8: como en ese modo no hay superficies rellenas que oculten nada, se ven simultáneamente las aristas del frente y las del fondo, y aparece una diagonal por cada una de las seis caras, que es lo que confirma que todas están formadas por dos triángulos y que ninguna quedó sin cubrir.

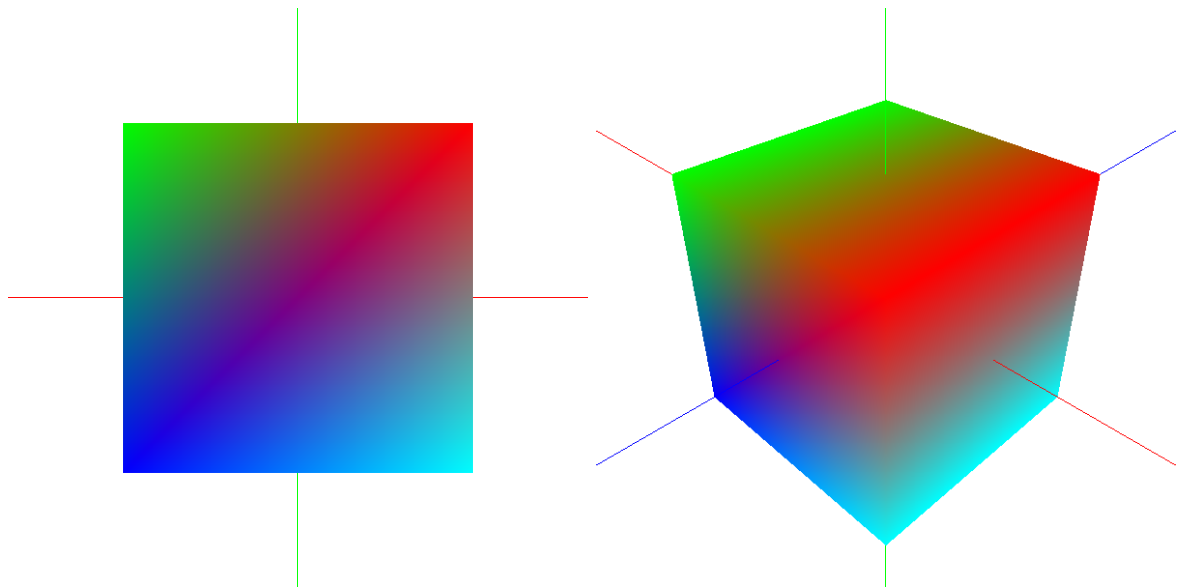


Figura 7: Las dos perspectivas del cubo de $2 \times 2 \times 2$: vista frontal con F1 (izquierda), en la que la figura es indistinguible de un cuadrado, y vista oblicua con F2 (derecha), en la que se aprecian tres caras.

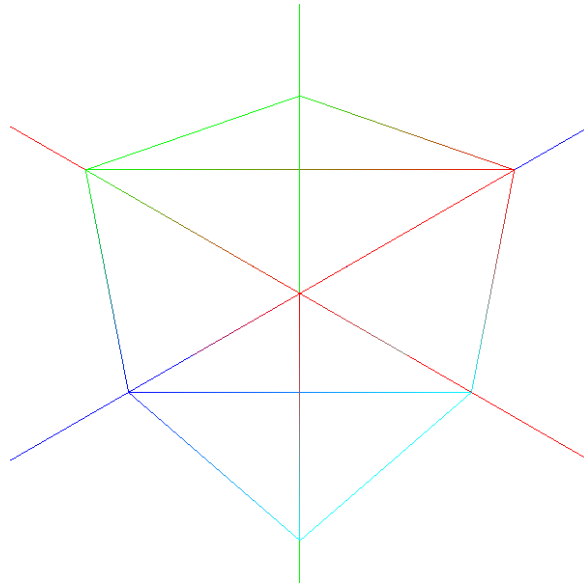


Figura 8: El cubo en modo de alambre desde la vista oblicua. Al no haber caras rellenas, se ven también las aristas del fondo y las diagonales de cada cara.

4.3. Actividad 3: Círculo de radio unitario

Esta actividad fue la que dio más material. Empezamos ejecutando la clase `Circle` tal como viene en el proyecto base y el resultado, que se muestra en la Figura 9, no fue el que esperábamos: en lugar del degradado continuo que produce la interpolación de colores entre vértices, el círculo apareció dividido en tres sectores de color plano, con transiciones abruptas justo en los 0° , 120° y 240° .

Al leer la clase encontramos la causa. El lazo asigna color únicamente a los vértices de esos tres ángulos y deja los demás sin escribir, y como `Vertex` es una estructura sin constructor, declararla dentro del lazo no inicializa sus miembros [10]. En nuestra compilación la variable ocupa siempre la misma posición de la pila, de modo que cada vértice hereda el color del anterior y el resultado son los tres sectores planos que se observan; es importante subrayar que se trata de un valor indeterminado y que con otro compilador, u otra configuración, la imagen podría ser distinta. Ése fue el experimento que más nos costó interpretar, porque durante un rato buscamos el problema en el shader y no en la construcción de los vértices.

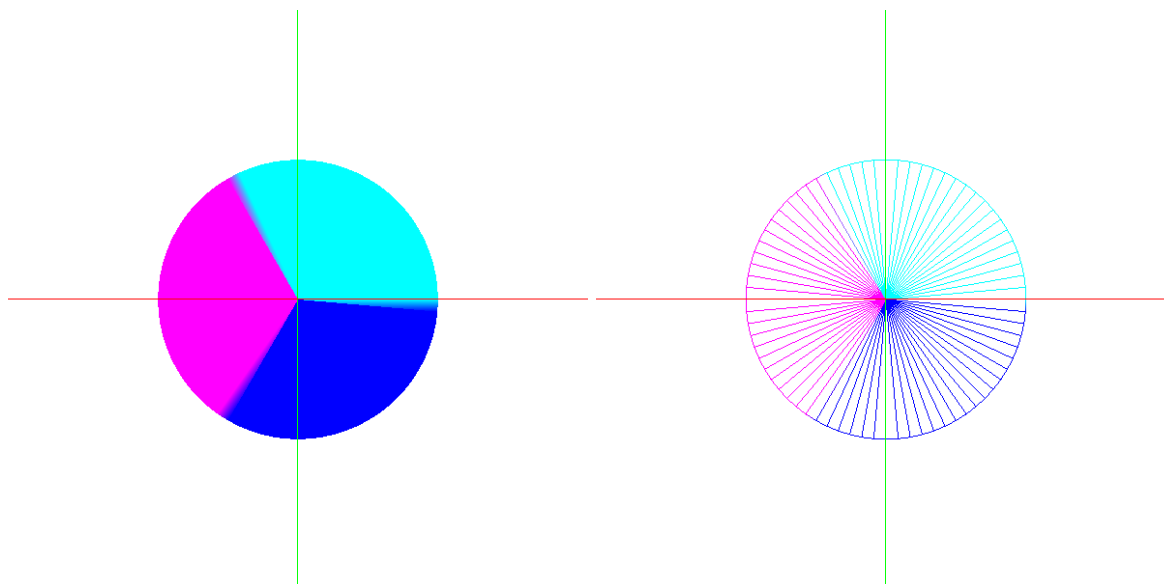


Figura 9: Resultado de la clase **Circle** del proyecto base: a la izquierda, los tres sectores de color plano que produce dejar sin inicializar el color de la mayoría de los vértices; a la derecha, la misma figura en modo de alambre, donde se aprecia que todos los triángulos del abanico convergen en el centro.

El segundo hallazgo no era visible en pantalla y apareció al contar los arreglos que genera la clase. Se instrumentó el programa para imprimir su tamaño y el índice más grande que emplea, con el resultado siguiente:

```
1 Circle base : 144 vertices , 435 indices , indice maximo = 144
2 Circle nueva: 73 vertices , 216 indices
```

Listing 16: Salida de la instrumentacion de las dos versiones de la clase **Circle**

Los dos números de la primera línea son problemáticos. Por un lado, la clase genera 144 vértices para una figura que necesita 73, porque duplica el vértice del centro una vez por cada punto del contorno; ese desperdicio explica también por qué su segundo lazo de índices no dibuja nada, ya que todos los triángulos que arma tienen dos de sus tres vértices en el centro y quedan degenerados. Por otro lado, y esto es lo grave, el índice más grande que utiliza es el 144, mientras que los índices válidos van del 0 al 143: la clase le pide a OpenGL un vértice que no existe. Este error no produjo ningún síntoma visible en nuestro equipo, lo cual lo vuelve más peligroso que el de los colores, porque nada en la imagen delata que está ahí.

Con la clase reescrita repetimos las pruebas. La Figura 10 muestra las dos vistas: con **F1** se obtiene el círculo y con **F2** la misma figura se proyecta como una elipse, que es justamente lo que debe ocurrir con una figura plana observada desde un ángulo oblicuo. El degradado ahora sí es continuo alrededor de todo el contorno.

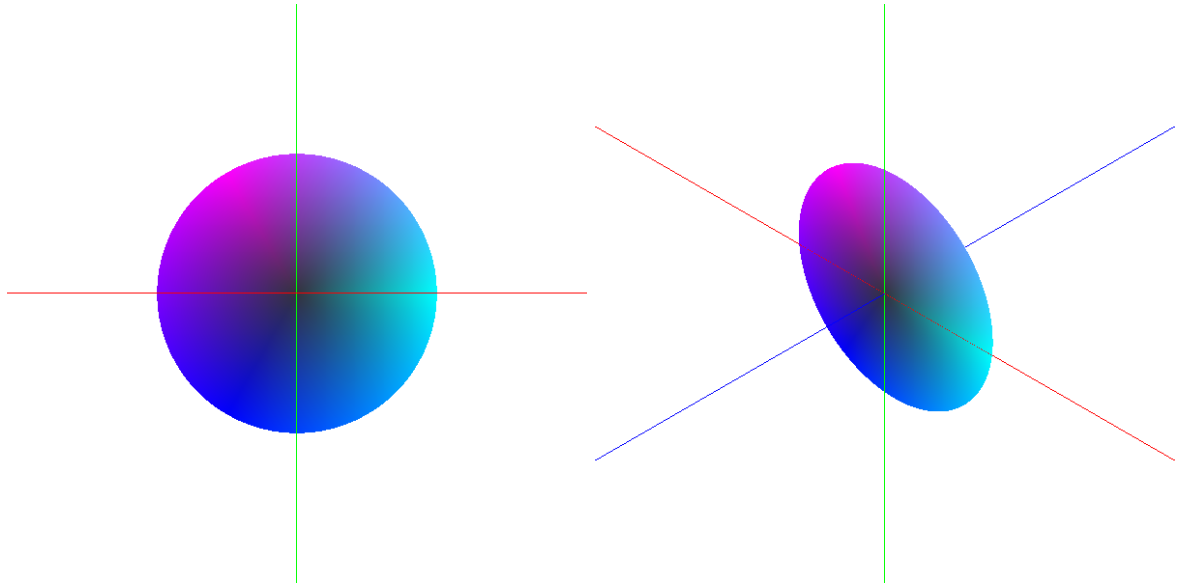


Figura 10: Las dos perspectivas del círculo de radio unitario con la clase reescrita: vista frontal con **F1** (izquierda) y vista oblicua con **F2** (derecha), en la que la figura se proyecta como una elipse.

El último experimento consistió en variar el número de lados del polígono con el que se aproxima la circunferencia. Con los 72 lados que quedaron como valor predeterminado, un vértice cada cinco grados, el contorno se ve liso a la resolución de la ventana; al bajarlo a 12 la figura deja de parecer un círculo y se reconoce con claridad como un polígono, tal como se aprecia en la Figura 11. El modo de alambre, en la Figura 12, deja ver el abanico completo: 72 triángulos que comparten el vértice del centro.

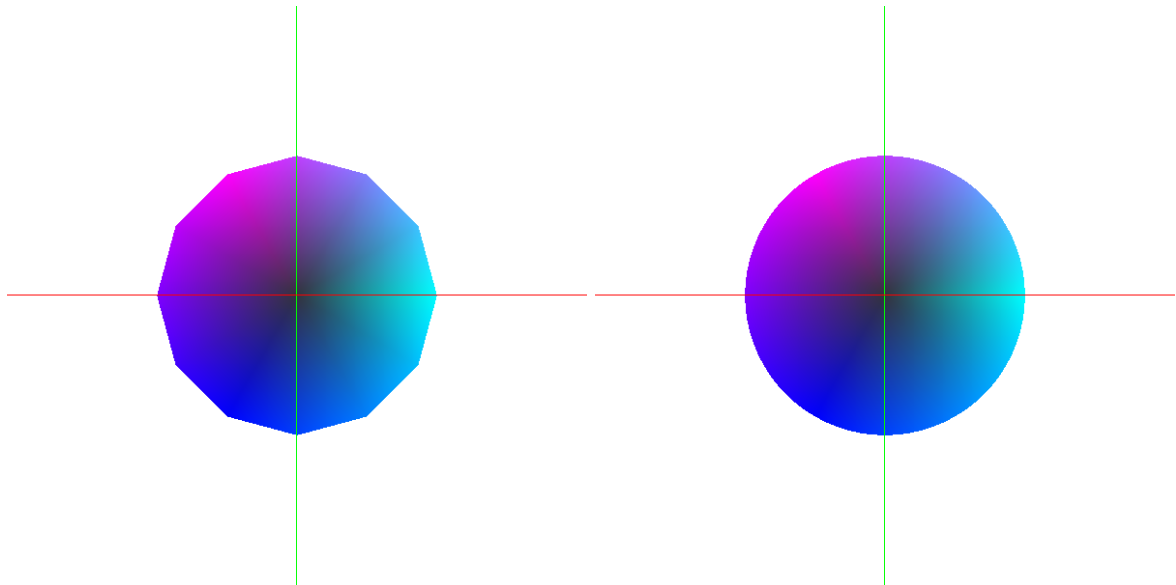


Figura 11: Efecto del número de lados sobre la aproximación de la circunferencia: con 12 lados el contorno es claramente poligonal (izquierda) y con 72 se percibe liso a la resolución de la ventana (derecha).

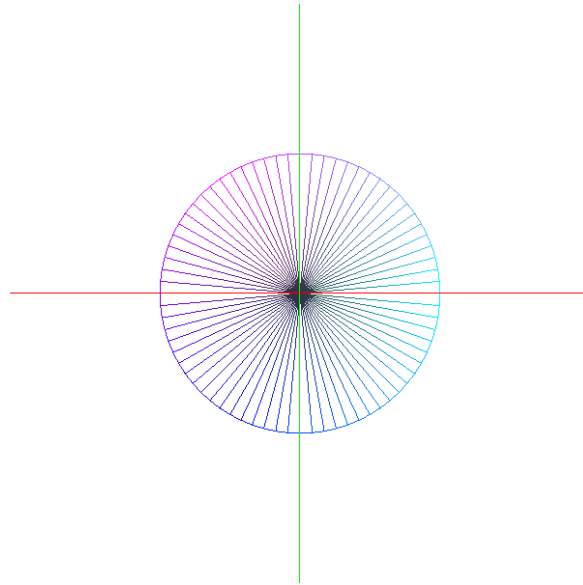


Figura 12: El círculo en modo de alambre: los 72 triángulos del abanico convergen en el único vértice del centro.

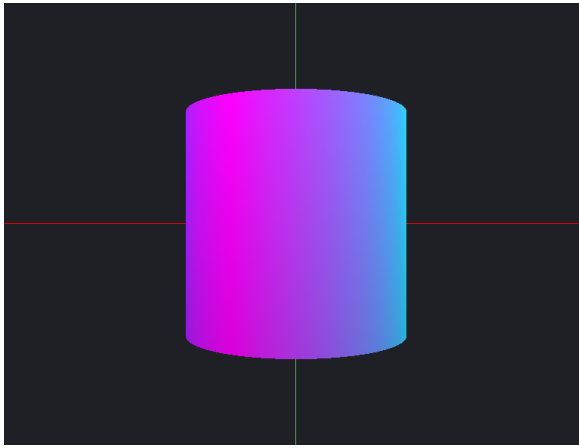
4.4. Actividad 4: Cilindro centrado en el origen

Para verificar el correcto modelado del cilindro, se realizaron pruebas de ejecución modificando las condiciones de la cámara y el modo de renderizado de OpenGL. Se configuraron las teclas de función F1 y F2 para alternar entre dos puntos de vista clave.

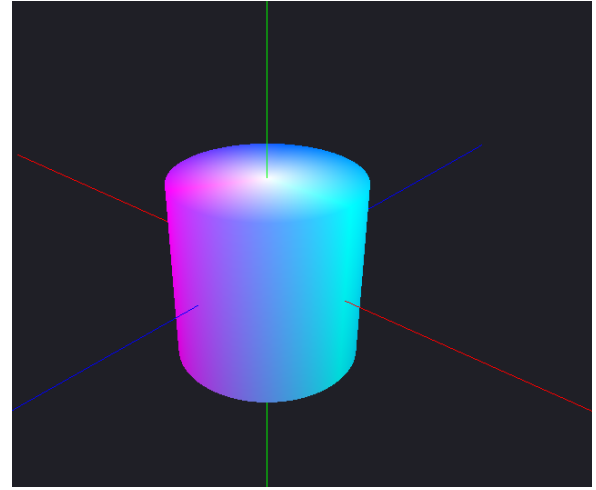
El experimento se dividió en las siguientes fases:

1. **Prueba de alineación y simetría (F1):** Se posicionó la cámara en $(0, 0, 5)$ con vista directa hacia el origen $(0, 0, 0)$. Esta perspectiva frontal permitió comprobar que el cuerpo del cilindro abarcara una unidad hacia arriba y una unidad hacia abajo sobre el eje vertical Y , y que sus costados coincidieran con $x = -1, 0$ y $x = +1, 0$.
2. **Prueba de volumen tridimensional y sellado (F2):** Se cambió la cámara a una vista oblicua en $(4, 0, 3, 5, 4, 5)$. Mediante esta toma se verificó que la tapa superior e inferior estuvieran construidas correctamente, y que la interpolación continua de color eliminara las caras negras del código original.

En la Figura 13 se ven las tres capturas obtenidas durante las pruebas.



(a) Vista frontal (F1).



(b) Vista oblicua (F2).

Figura 13: Pruebas experimentales del cilindro de radio 1 y altura 2 centrado en el origen.

4.5. Actividad 5. Esfera centrada en el origen

Con la triangulación ya implementada, el primer experimento reveló el mismo patrón de color que ya habíamos documentado en la Actividad 3: al dejar el esquema de color original de la clase, sólo los tres meridianos en $\theta = 0^\circ$, 120° y 240° se veían coloreados, y el resto de la superficie aparecía en negro, con la triangulación ya correcta pero invisible al no recibir color propio. La Figura 14 lo muestra en las dos vistas.

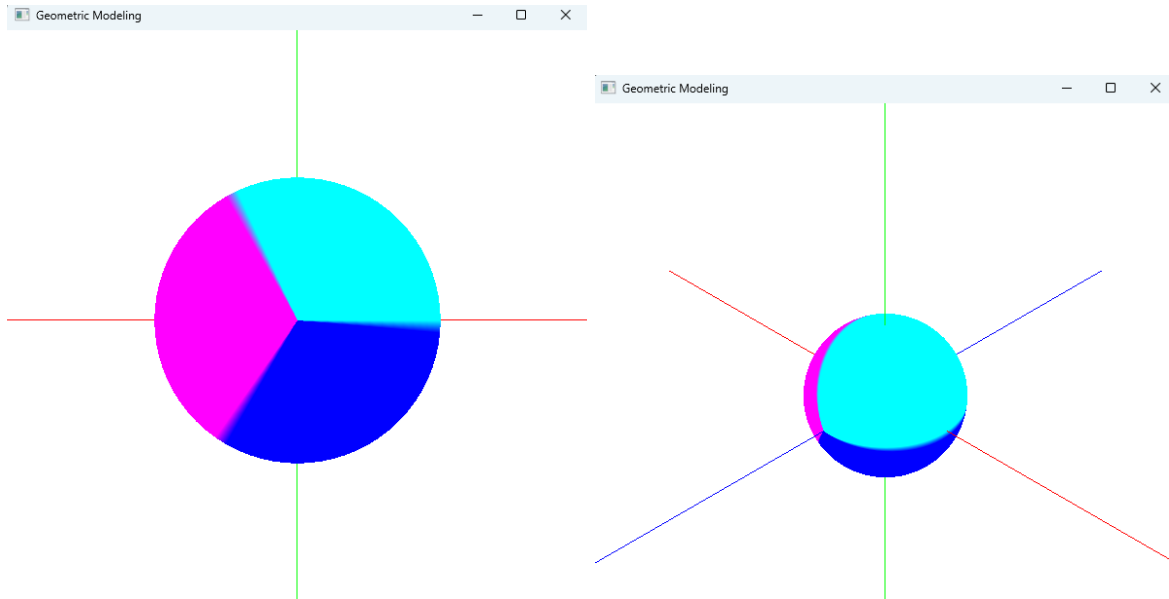


Figura 14: Esfera ya triangulada pero con el esquema de color original de la clase: sólo tres meridianos reciben color y el resto de los 2592 vértices quedan sin inicializar. Vista frontal con F1 (izquierda) y oblicua con F2 (derecha).

Con el color corregido mediante el mapeo de la posición normalizada, repetimos las pruebas de siempre. Con F1 y F2 se confirmó que la esfera se ve igual de redonda desde cualquier ángulo, a diferencia del plano y el círculo, cuya silueta sí cambiaba entre una vista y otra; esto tiene sentido porque la silueta de una esfera es un círculo sin importar la dirección desde la que se observe. La Figura 15 muestra las dos vistas con el degradado ya continuo.

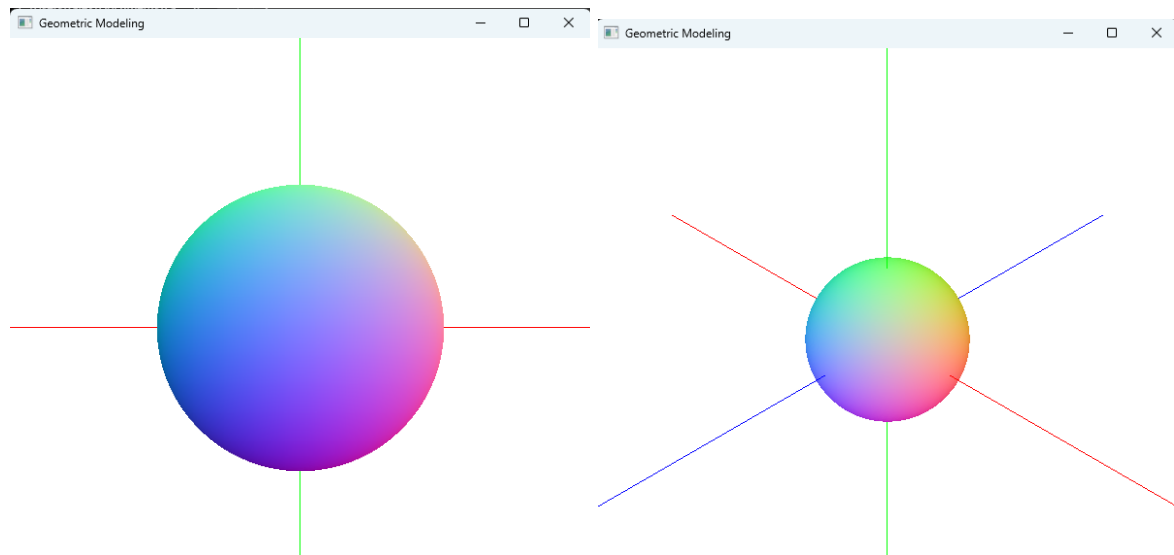


Figura 15: Las dos perspectivas de la esfera de radio unitario, ya con el color corregido: vista frontal con F1 (izquierda) y vista oblicua con F2 (derecha).

4.6. Actividad 6: Cono centrado en el origen

Para esta actividad, primero corrimos la clase del cono tal como venía en la plantilla que nos dieron. Nos pasó exactamente lo mismo que con el círculo: casi toda la figura se veía de color negro y sólo tenía tres líneas de color en 0° , 120° y 240° . Al revisar el código vimos que a los demás puntos no se les había puesto color, así que la computadora no sabía de qué tono pintarlos. Además, al girar la cámara por debajo, la figura estaba completamente hueca porque le faltaba la tapa de abajo.

También revisamos los números del código y notamos que repetía la punta del cono 72 veces en el mismo lugar, gastando memoria y armando triángulos encimados que ni se veían. Por eso arreglamos la clase para ponerle un solo punto en la punta, cerrar la base redonda con su propia tapa y repartir bien los colores cian, magenta y azul.

Con la figura ya corregida, probamos las diferentes vistas del programa, las cuales se muestran en la Figura 16:

1. Con la tecla F1 pusimos la vista de frente. Como la cámara queda derecha, el cono se ve perfectamente como un triángulo plano.
2. Con la tecla F2 pasamos a la vista de lado en diagonal. Aquí sí se nota el volumen en 3D y se comprueba que el cono está justo en medio del origen, ya que las líneas de los ejes le pasan por el centro.
3. Con la tecla L activamos el modo de alambre. Así pudimos comprobar que todos los triángulos suben parejitos hacia la punta y que el círculo de la base quedó bien cerrado sin huecos.

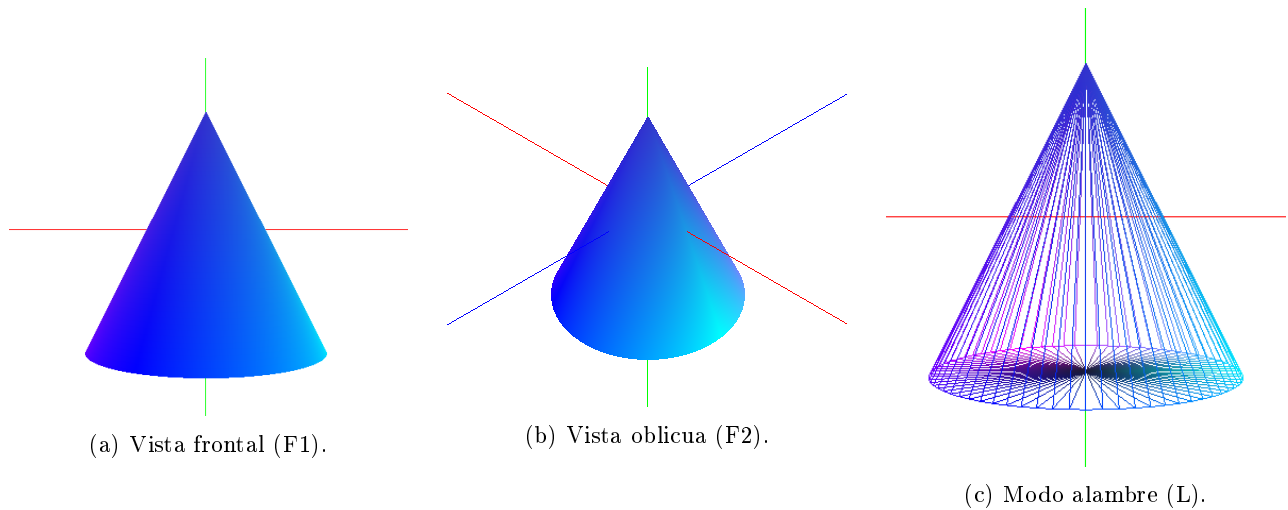


Figura 16: Pruebas de visualización del cono de radio 1 y altura 2 centrado en el origen.

4.7. Actividad 7: Escena con OpenGL

Para los experimentos se modificaron únicamente las transformaciones y los colores de algunos objetos de la escena, cambiando un solo objeto por cada tipo de transformación para poder compararlos contra el resultado original. Los cuatro cambios se aplicaron al mismo tiempo y se pueden ver en la Figura 17.

En el primer experimento se cambió el escalamiento de la esfera roja de $(0,25, 0,25, 0,25)$ a $(0,25, 0,25, 0,5)$ y su centro se subió de $y = 0,25$ a $y = 0,5$ para que siguiera apoyada en el piso. La esfera se convirtió en un óvalo alargado de forma vertical, aunque el escalamiento se hizo en el eje z . Esto pasa porque el escalamiento se aplica antes que la rotación de 90° respecto al eje x , y esa rotación convierte el eje z local de la esfera en el eje vertical de la escena, lo que confirma que el orden $T R S$ de las transformaciones sí afecta el resultado.

En el segundo experimento se cambió la rotación del cono grande de -90° a -60° respecto al eje x . El techo dejó de estar derecho sobre la casa y quedó inclinado 30° , por lo que ahora se alcanza a ver parte de su base y ya no cubre de forma pareja al cilindro.

En el tercer experimento se trasladó el tercer árbol de $x = -2$ a $x = 2$, moviendo tanto el tronco como las hojas con los mismos valores en x y z . El árbol quedó del lado derecho de la escena, frente a la casa, sin separarse. Esto se debe a que el tronco y las hojas son objetos independientes con su propia matriz de modelo, así que para mover el árbol completo fue necesario aplicar la misma traslación a los dos.

Por último, en el cuarto experimento se cambiaron los colores de la casa: el cilindro pasó de naranja barro a azul grisáceo, con $(0,35, 0,45, 0,60)$, y el techo de beige a rojo teja, con $(0,70, 0,15, 0,10)$. El cambio se reflejó sin afectar al resto de los objetos, ya que cada uno tiene su propia malla con los colores asignados en sus vértices.

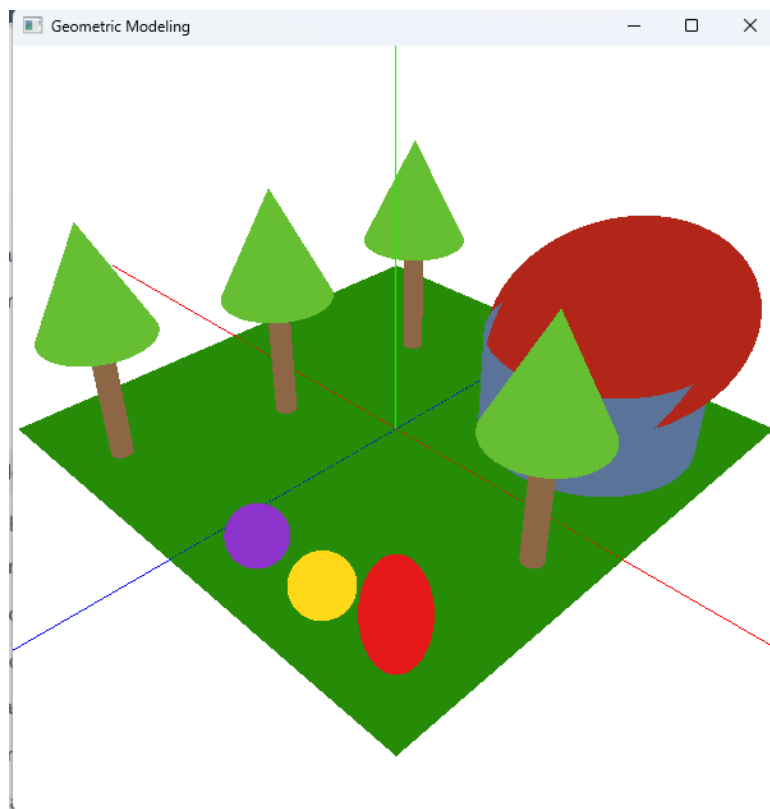


Figura 17: Escena con los cuatro experimentos aplicados: esfera roja escalada, techo rotado -60° , tercer árbol trasladado a la derecha y nuevos colores de la casa, vista oblicua (F2).

5. Resultados

En la Tabla 3 se reúnen las tres primitivas de esta entrega con las medidas que pide el manual, la llamada con la que se construyen y el costo que tiene cada una en vértices y en triángulos. Los tres objetos quedan centrados en el origen y se seleccionan en tiempo de ejecución con las teclas 1, 2 y 3.

Tecla	Primitiva	Construcción	Vértices	Triángulos
1	Plano de 2×2 paralelo a XY	<code>Plane(2.0f)</code>	4	2
2	Cubo de $2 \times 2 \times 2$	<code>Cube(2.0f)</code>	8	12
3	Círculo de radio 1	<code>Circle(1.0f, 72)</code>	73	72

Tabla 3: Resumen de las tres primitivas implementadas, con su costo en vértices y triángulos.

5.1. Actividad 1: Plano paralelo al plano XY

La versión final construye el plano con `Plane(2.0f)`, con lo que sus cuatro vértices caen en $(\pm 1, \pm 1, 0)$ y la figura mide las 2×2 unidades solicitadas. El resultado se muestra en la Figura 18 en las dos vistas que pide el manual.

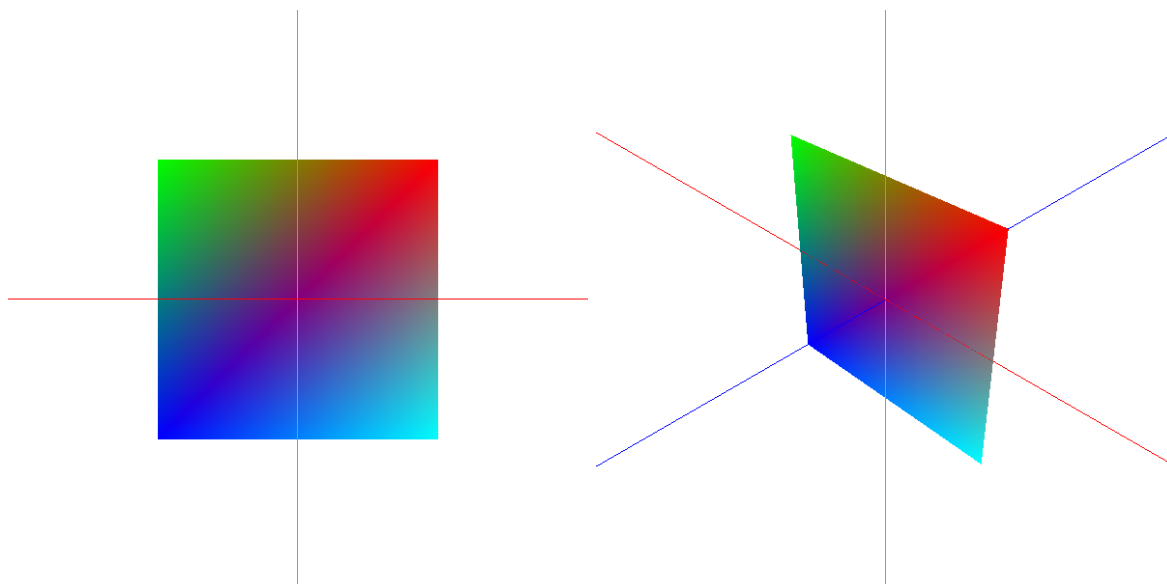


Figura 18: Resultado final de la Actividad 1: el plano de 2×2 unidades centrado en el origen, visto de frente con F1 (izquierda) y desde un ángulo oblicuo con F2 (derecha).

La ventaja de esta solución es que no hizo falta escribir código nuevo: bastó con leer con cuidado el constructor de la clase que ya venía en el proyecto y elegir el argumento correcto. Que el constructor reciba el lado completo, y no el semilado, es además lo que garantiza que la figura quede centrada en el origen para cualquier tamaño, sin necesidad de aplicar después una traslación con la matriz de modelo.

Como desventaja, el plano tiene una sola cara en el sentido de que no se le asignó normal ni se activó el descarte de caras traseras, de modo que se ve igual por delante que por detrás; eso es suficiente para esta práctica pero dejará de serlo cuando se agreguen modelos de iluminación. La segunda limitación es la que apareció en los experimentos: al estar contenido en $z = 0$, el plano comparte profundidad exacta con los ejes coordenados y las líneas de los ejes quedan por encima de él. Es un artificio del arreglo particular de esta escena y no un error de la figura, pero conviene tenerlo presente, porque la única forma de evitarlo sería desplazar ligeramente los ejes o desactivar la escritura de profundidad al dibujarlos.

5.2. Actividad 2: Cubo centrado en el origen

El cubo final se construye con `Cube(2.0f)`, con los ocho vértices en $(\pm 1, \pm 1, \pm 1)$ y las seis caras formadas por doce triángulos. La Figura 19 muestra las dos vistas solicitadas.

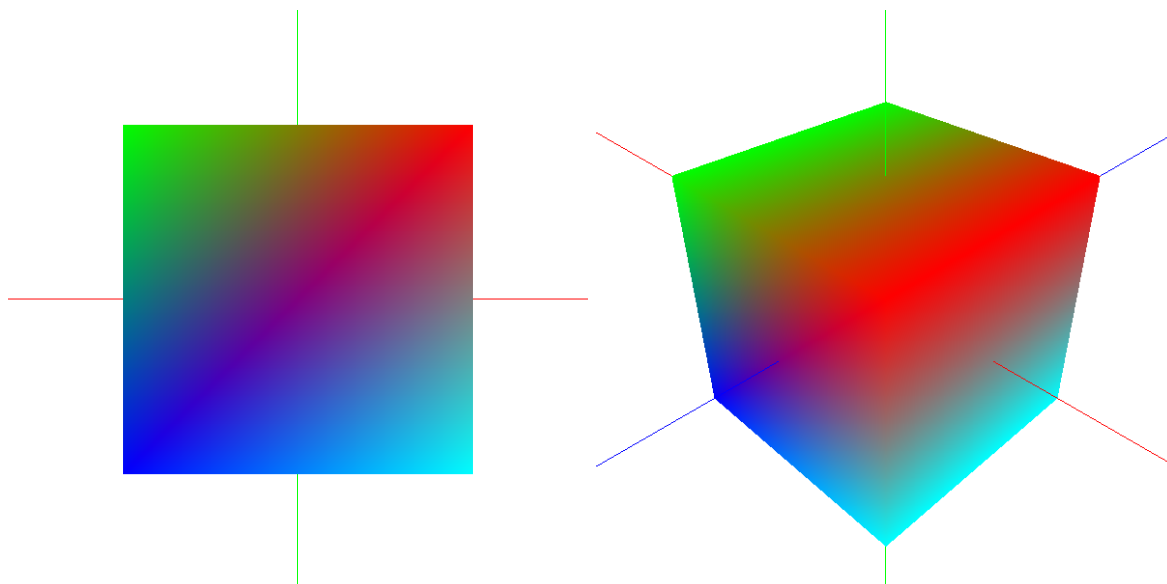


Figura 19: Resultado final de la Actividad 2: el cubo de $2 \times 2 \times 2$ unidades centrado en el origen, visto de frente con F1 (izquierda) y desde un ángulo oblicuo con F2 (derecha).

La ventaja principal de la construcción es su economía: ocho vértices bastan para las seis caras porque cada uno participa en tres de ellas, y esa reutilización es precisamente lo que permite el arreglo de índices. El degradado que se observa sobre las caras no costó ningún trabajo adicional; es el resultado de que cada vértice lleve su propio color y de que la etapa de rasterización interpole los valores intermedios.

Esa misma economía tiene dos consecuencias que conviene anotar. La primera es que, al compararse los vértices, es imposible darle a cada cara un color plano distinto sin duplicar geometría: habría que pasar de ocho a veinticuatro vértices, tres copias de cada esquina. La segunda es que el sentido de recorrido de los índices no es consistente entre las seis caras del proyecto base; mientras no se active `glEnable(GL_CULL_FACE)` el resultado es idéntico, pero en cuanto se active, algunas caras desaparecerán y habrá que reordenar los índices. Lo dejamos así a propósito, porque corregirlo ahora hubiera hecho más difícil comparar nuestra versión con la del proyecto base, pero queda identificado como trabajo pendiente.

5.3. Actividad 3: Círculo de radio unitario

El círculo final se construye con la clase reescrita, invocada como `Circle(1.0f, 72)`. La Figura 20 muestra las dos vistas, y en la Tabla 4 se comparan las dos versiones de la clase con los números que arrojó la instrumentación del programa.

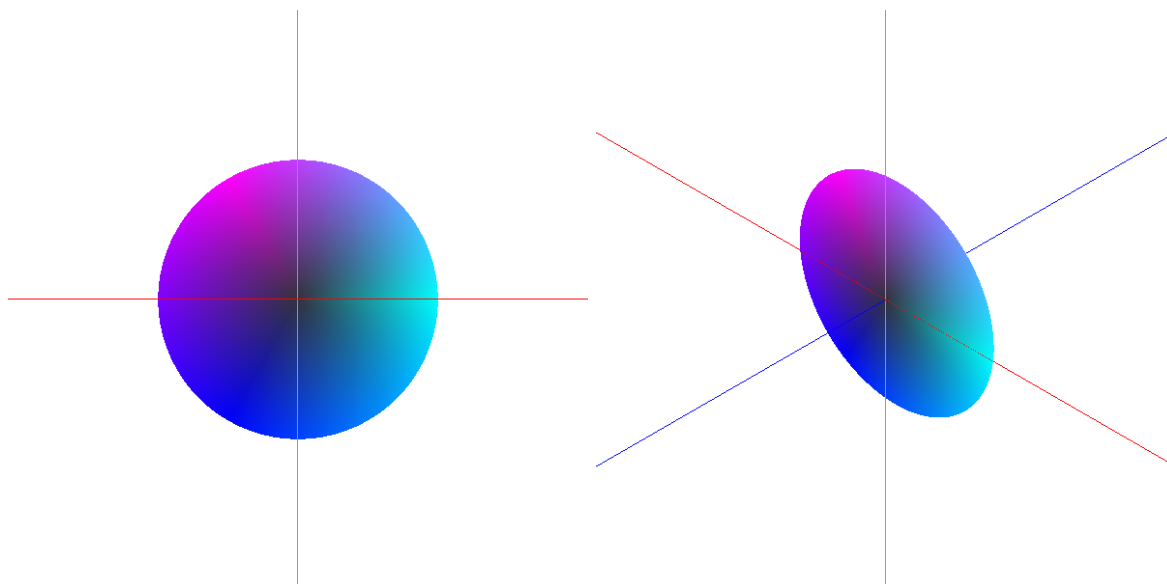


Figura 20: Resultado final de la Actividad 3: el círculo de radio unitario centrado en el origen, visto de frente con F1 (izquierda) y desde un ángulo oblicuo con F2 (derecha), donde se proyecta como una elipse.

Aspecto	Clase del proyecto base	Clase reescrita
Vértices generados	144	73
Índices generados	435	216
Triángulos con área	72	72
Triángulos degenerados	73	0
Índice más grande empleado	144	72
Índice máximo válido	143	72
Vértices con color asignado	6 de 144	73 de 73
Número de lados	fijo en 72	parámetro del constructor

Tabla 4: Comparación entre la clase `Circle` del proyecto base y la versión reescrita.

La ventaja de la versión reescrita no es sólo que corrige los dos defectos. Al volver el número de lados un parámetro del constructor, la figura deja de ser un objeto fijo y se convierte en una familia de objetos, y esa es justamente la propiedad que se va a necesitar en las actividades siguientes: la tapa del cilindro y la base del cono son este mismo círculo con otro radio y otra posición. Además, el arreglo de índices quedó a la mitad y sin triángulos degenerados, lo que significa que la mitad del trabajo que hacía la versión original se gastaba en dibujar triángulos sin área.

La desventaja es que el número de lados quedó fijado en el código en el momento de la construcción, de modo que cambiarlo obliga a recompilar; para un ejercicio de esta escala no vale la pena, pero en un programa real convendría reconstruir la malla en tiempo de ejecución. Una segunda limitación es que la aproximación poligonal se vuelve visible en cuanto la cámara se acerca lo suficiente: el número de lados que basta a una distancia deja de bastar a otra, y resolverlo bien exigiría ajustar la teselación según el tamaño que ocupa la figura en pantalla, que es un tema que queda fuera del alcance de esta práctica.

5.4. Sobre la metodología empleada

Dos decisiones de método resultaron útiles más allá de las actividades concretas. La primera fue reunir el dibujo de las cuatro mallas en una sola función en lugar de repetir el bloque de código una

vez por figura, como venía en el proyecto base: además de acortar el archivo, garantiza que las tres primitivas se dibujen exactamente con la misma cadena de transformaciones, lo cual es indispensable para que las comparaciones de tamaño entre una y otra tengan sentido.

La segunda fue agregar el modo de alambre. En las tres actividades hubo al menos una comprobación que sólo pudo hacerse con esa tecla: contar los dos triángulos del plano, verificar que al cubo no le faltara ninguna cara y ver el abanico completo del círculo. En modo relleno, ninguna de esas tres cosas se distingue, porque la silueta de la figura es la misma esté bien o mal construida por dentro.

La limitación de la metodología es que todas las verificaciones fueron visuales, salvo la del conteo de vértices e índices. El caso del índice fuera de rango de la clase `Circle` original mostró bien el problema: es un error real que no produjo ningún efecto en pantalla y que sólo apareció al imprimir los valores del arreglo. Un método más sólido incluiría comprobaciones automáticas sobre los arreglos generados, por ejemplo verificar que todos los índices caigan dentro del rango válido y que ningún triángulo tenga dos vértices repetidos, antes de enviar la malla a la unidad de procesamiento gráfico.

5.5. Actividad 4: Cilindro centrado en el origen

La versión final del cilindro se instanció con los parámetros `Cylinder(1.0f, 2.0f, 72)`, logrando una geometría sólida, cerrada y centrada en el origen $(0,0,0)$. La estructura final quedó compuesta por un total de 146 vértices (72 por cada anillo perimétrico más 2 vértices centrales para las tapas) y 288 triángulos (144 para el cuerpo lateral y 72 en cada extremo plano). En la Figura 21 se aprecian las dos vistas finales.

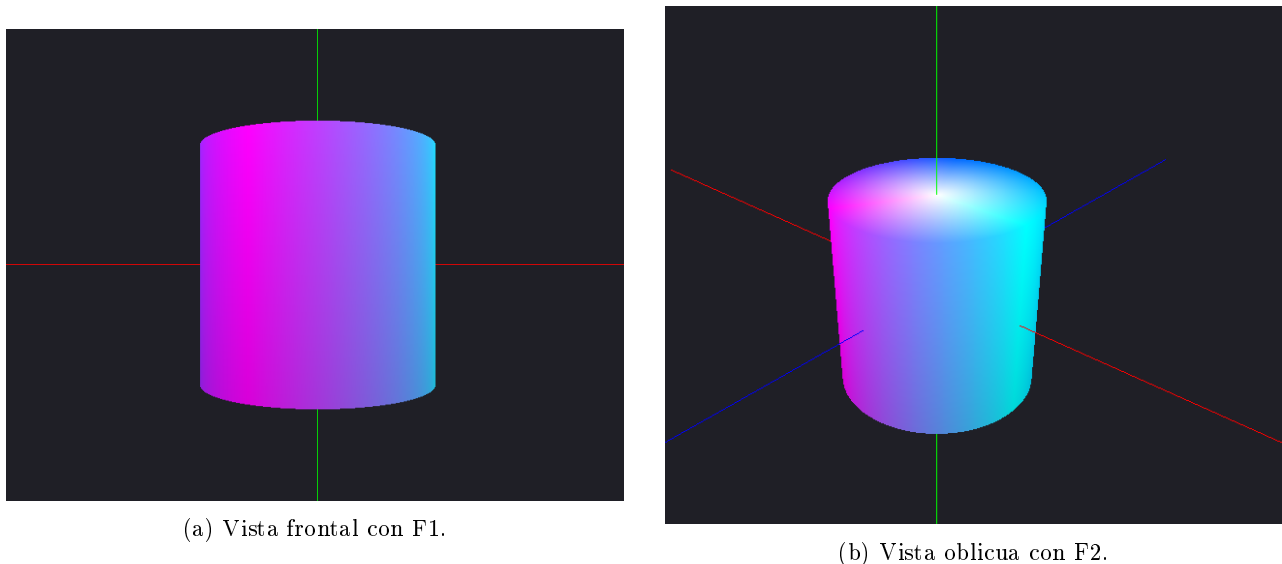


Figura 21: Resultado final de la Actividad 4: cilindro de radio 1 y altura 2 centrado en el origen, visto de frente (izquierda) y en perspectiva oblicua (derecha).

Al ver el cilindro de frente con la tecla F1 (Figura 21a), se ve como un rectángulo de 2 unidades de ancho por 2 de alto, lo que confirma que quedó derecho y bien centrado en el origen porque los ejes pasan justo por en medio. Al cambiar a la vista en diagonal con F2 (Figura 21b), se aprecia claramente su forma redonda y se comprueba que la tapa de arriba quedó bien cerrada.

La ventaja de este método es que poner el cilindro a lo largo del eje Y y usar un solo punto en el centro para tapar las bases hace que la figura quede completa sin gastar memoria de más. Como limitación, dejar el círculo fijo en 72 lados hace que se vea bien y redondito a una distancia normal, pero si acercamos mucho la cámara se alcanzan a notar las líneas rectas de los triángulos.

5.6. Actividad 5. Esfera centrada en el origen

La versión final se construye con `Sphere(1.0f)`, que genera 2592 vértices y, con la triangulación implementada, 5040 triángulos (35 filas de celdas por 72 columnas, dos triángulos por celda). La Figura 22 muestra el resultado final en las dos vistas.

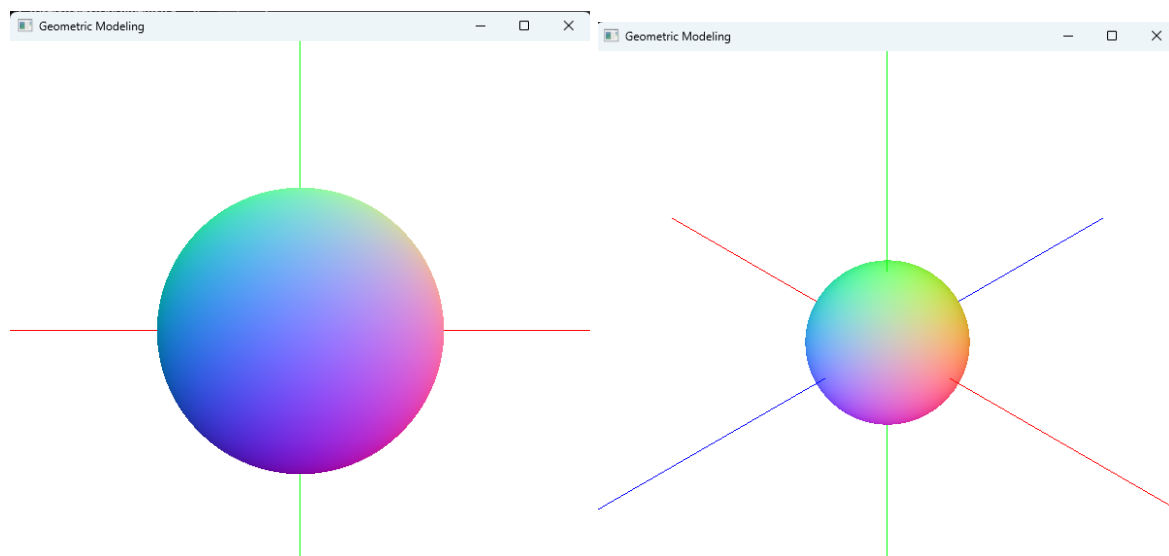


Figura 22: Resultado final de la Actividad 5: la esfera de radio unitario centrada en el origen, con el color asignado según la posición normalizada de cada vértice, vista de frente con F1 (izquierda) y desde un ángulo oblicuo con F2 (derecha).

La ventaja principal de resolver `buildIndices` por celdas de la rejilla, en lugar de caso por caso como en el cubo, es que el mismo esquema –dos filas consecutivas, dos columnas consecutivas, dos triángulos, cerrar la columna con el operador módulo– no depende de que la figura sea específicamente una esfera: cualquier superficie generada por revolución sobre una rejilla de φ y θ se triangula igual, así que este mismo código es, en esencia, el que se reutilizará en el cilindro y en el cono de las actividades 4 y 6, cambiando únicamente la fórmula con la que se calcula la posición de cada vértice. El color por posición normalizada también resultó más barato que la interpolación por anclas del círculo, porque no exige guardar un arreglo de colores de referencia ni calcular en qué sector cae cada ángulo.

Como desventaja, los ángulos de paso quedaron fijos en 5° dentro del propio método, y no como parámetro del constructor; siguiendo el mismo criterio que se aplicó al reescribir el círculo, sería preferible exponer el número de anillos y de columnas como argumentos, de modo que la resolución de la esfera pueda ajustarse sin recompilar.

5.7. Actividad 6: Cono centrado en el origen

Para el resultado final, creamos el cono usando la instrucción `Cone(1.0f, 2.0f, 72)`, logrando que mida 1 unidad de radio y 2 unidades de alto, quedando justo en medio del origen de coordenadas. En total se usaron 74 puntos (la punta de arriba, 72 puntos alrededor de la orilla y uno más en el centro del círculo de abajo) y 144 triángulos (72 para el cuerpo inclinado y 72 para tapar la base). En la Figura 23 se pueden ver las dos tomas.

Al poner la vista de frente con la tecla F1 (Figura 23a), el cono se ve derecho como un triángulo plano que mide una unidad hacia arriba y una hacia abajo del centro. Cuando cambiamos a la vista en diagonal con F2 (Figura 23b), ya se nota redondito en 3D y se comprueba que no quedó hueco de abajo.

La ventaja principal de esta forma de trabajar fue que dejamos la figura bien programada desde cero: en vez de dejar valores fijos, le dejamos variables para el radio, la altura y los lados, así que si

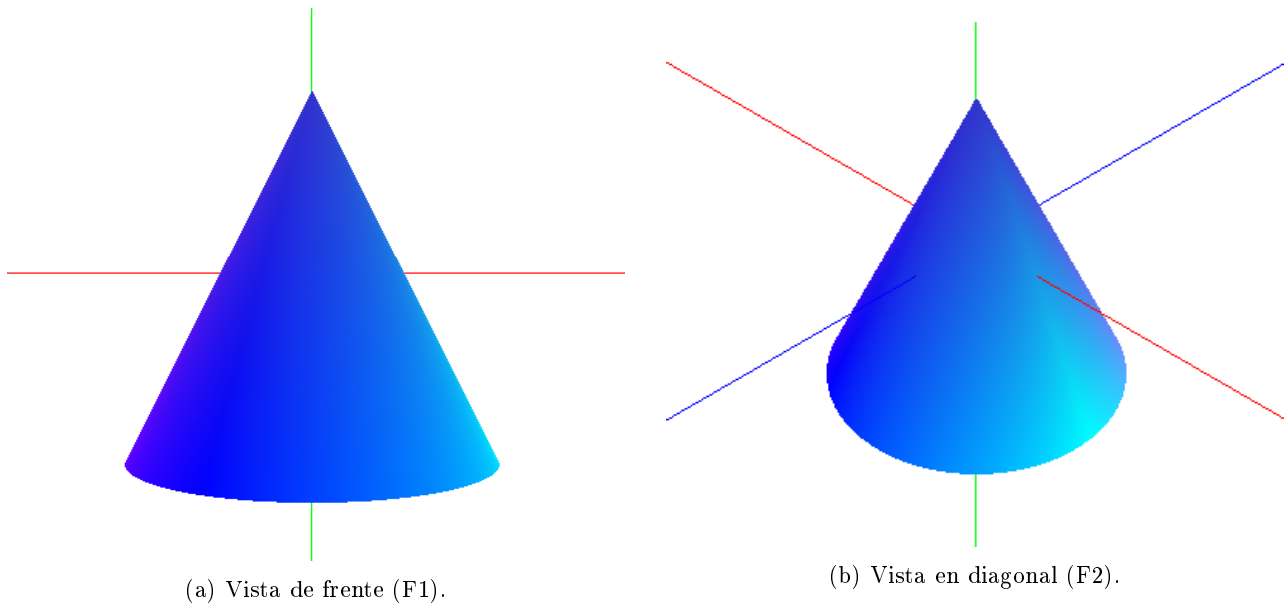


Figura 23: Resultado final de la Actividad 6: cono de radio 1 y altura 2 centrado en el origen, visto de frente (izquierda) y de lado (derecha).

quisiéramos cambiarle el tamaño a futuro se hace muy fácil. También evitamos que la computadora guardara la punta 72 veces en el mismo lugar, ahorrando memoria y quitando triángulos encimados que no servían de nada.

Como desventaja, el número de lados se queda fijo en el código; si acercáramos mucho la cámara, la orilla redonda empezaría a verse recta como un polígono y haría falta aumentarle las divisiones. Además, todavía no le calculamos normales a las caras, lo cual ahorita se ve bien con los colores básicos, pero hará falta agregárselo más adelante cuando nos toque poner luces y sombras en la escena.

5.8. Actividad 7: Escena con OpenGL

Como resultado se obtuvo una escena con todos los objetos apoyados sobre la superficie, que quedó de color verde con sus colores originales. Del lado derecho se formó una casa con el cilindro naranja barro como base y el cono beige como techo, del lado izquierdo quedaron los cuatro árboles formados por los troncos café claro y los conos verde claro, y al frente las tres esferas alineadas en rojo, amarillo y morado. En la vista oblicua se aprecia la distribución completa de la escena sobre el piso, mientras que en la vista frontal el plano no se alcanza a ver porque la cámara queda a la misma altura que él, y las esferas aparecen encimadas sobre la casa porque están más cerca de la cámara. En la Figura 24 se pueden ver las dos tomas.

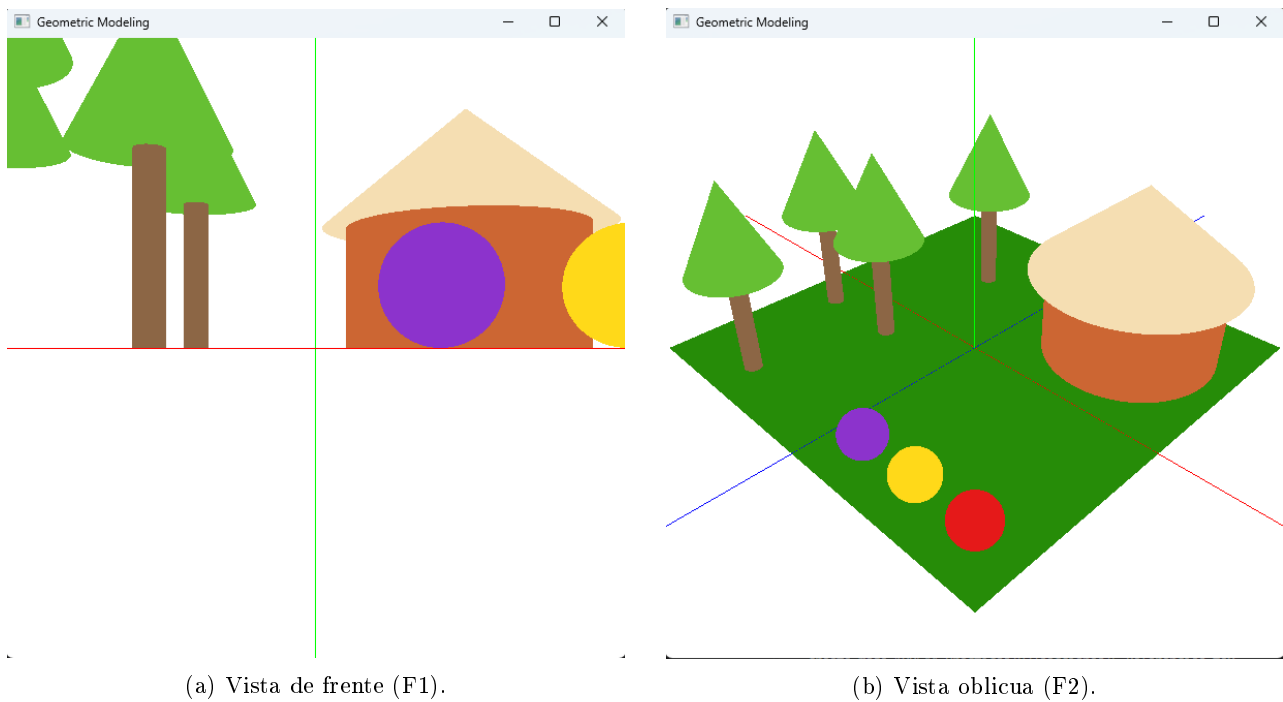


Figura 24: Resultado final de la Actividad 7: escena construida con el plano, cilindros, esferas y conos, vista de frente (izquierda) y en diagonal (derecha).

6. Conclusiones

6.1. Basilio Illescas Andrés

Los objetivos de esta entrega se cumplieron: las tres primeras actividades del manual quedaron implementadas y verificadas, con el plano de 2×2 , el cubo de $2 \times 2 \times 2$ y el círculo de radio unitario centrados en el origen y observables desde las dos perspectivas asignadas a F1 y F2. Lo que realmente me llevo es que el modelado no consiste en escribir coordenadas sino en elegir una descomposición: el plano sólo pedía leer bien el constructor, el cubo pedía verificar por caras que no faltara ninguna, y el círculo pedía decidir cómo se aproxima una curva y con cuántos lados, de modo que la figura dejó de escribirse y pasó a generarse. La lección de método vino de los dos errores de la clase `Circle` del proyecto base: el de los colores se veía de inmediato, pero el índice fuera de rango no produjo ningún síntoma en pantalla y sólo apareció al imprimir los arreglos, lo que me dejó claro que en gráficos «se ve bien» no equivale a «está bien» y que por eso valió la pena agregar la tecla que alterna entre relleno y alambre.

6.2. Cruz Macedo Samuel Santiago

Al principio esta práctica me pareció más sencilla que la anterior, porque ya no había que pelearse con tres matrices, pero la dificultad simplemente se movió de lugar: en la Práctica 02 el problema era decidir desde dónde se mira la figura y aquí es decidir de qué está hecha. Lo que más me sirvió fue entender la diferencia entre la primitiva que uno imagina y la que la tarjeta dibuja, porque no existe una instrucción para dibujar un círculo: son setenta y dos triángulos que comparten el centro, y al bajar el número de lados a doce la figura se volvió un polígono evidente, lo que deja ver que toda curva en gráficos es una aproximación. De la clase `Circle` del proyecto base me quedó una lección aparte, ya que los tres sectores de color plano que salían no eran culpa del shader sino de que `Vertex` no se inicializa sola: fue la primera vez que vi un programa correr sin marcar error y aun así entregar algo indefinido. Considero que los objetivos se cumplieron, y me queda claro que esto es la base de lo que

sigue, porque el cilindro y el cono son este mismo círculo repetido o unido a un punto.

6.3. Medel Piña Ángel de Jesús

Con esta práctica pude comprender cómo se construye una escena a partir de figuras básicas y cómo las transformaciones geométricas permiten reutilizar una misma geometría varias veces. Al armar la escena noté que el orden en que se aplican la traslación, la rotación y el escalamiento sí importa, ya que al multiplicarse como $T R S$ primero se escala la figura en sus propios ejes, luego se orienta y al final se coloca en su posición. Gracias a esto pude, por ejemplo, recortar el cilindro y el cono en el eje z para ajustar su altura antes de rotarlos y dejarlos de pie sobre el piso, además de calcular la altura de cada objeto para que quedara apoyado sobre la superficie en lugar de atravesarla o quedar flotando.

También me di cuenta de que el color de las figuras no es una propiedad que se pueda cambiar al momento de dibujarlas, sino que forma parte de los datos de cada vértice que se envían al *buffer* al crear la malla. Por eso, para tener objetos del mismo tipo con colores distintos fue necesario construir una malla por cada color, mientras que para repetir un objeto con el mismo color bastaba con dibujar la misma malla con diferentes matrices de modelo. Por último, implementar las caras de la esfera me ayudó a entender cómo una rejilla de vértices guardada en un solo arreglo se puede convertir en triángulos calculando directamente sus índices, y cómo el operador módulo permite cerrar la figura sin dejar costuras abiertas.

6.4. Perez Olvera Alexis Abraham

Esta práctica me dejó claro que la parte difícil de construir una geometría 3D no son las fórmulas matemáticas, que casi siempre ya venían resueltas en el proyecto base, sino los detalles que uno da por sentados: inicializar bien una estructura, cerrar un contorno, no repetir un vértice de más. El caso del índice fuera de rango en el círculo, que no producía ningún error visible en pantalla, fue el que más me hizo entender que una figura puede compilar y verse bien sin estar realmente correcta, y que conviene desconfiar un poco de lo que se ve bien a simple vista.

También me quedó una intuición más clara de algo que antes usaba sin pensarlo demasiado: cualquier modelo 3D, por sofisticado que parezca, es en el fondo lo mismo que hice aquí a mano —vértices, colores y triángulos que los conectan. Verlo funcionar, con sus errores incluidos, terminó explicándome esa idea mejor que cualquier lectura previa.

6.5. Segura Cedeño Luisa María

Esta práctica me ayudó a entender cómo se construyen figuras tridimensionales más complejas a partir de triángulos básicos. Al principio pensé que bastaba con usar el archivo del cono que ya venía en el proyecto base, pero al correrlo me di cuenta de que tenía varios problemas: casi todo se veía negro porque le faltaba ponerle color a la mayoría de los puntos, la punta de arriba estaba repetida muchas veces y la figura quedaba hueca por no tener tapa abajo.

Al ver el trabajo completo con el plano, el cubo, el círculo, la esfera y el cono, entendí que toda geometría tridimensional, sin importar qué tan redonda parezca, termina siendo una descomposición en triángulos y vértices ordenados. Además, poder alternar entre las vistas y usar el modo de alambre fue indispensable para comprobar que ninguna cara quedara mal conectada, demostrando que en gráficos por computadora no basta con que algo se vea bien a simple vista, sino que su estructura interna debe estar bien construida.

Referencias

- [1] Microsoft Learn. (s.f.). *Control gráfico OpenGL - Win32 apps*. Recuperado de <https://learn.microsoft.com/es-es/windows/win32/opengl/introduction-to-opengl>

- [2] Microsoft Learn. (s.f.). *Modo RGBA y administración de paletas de Windows - Win32 apps*. Recuperado de <https://learn.microsoft.com/es-es/windows/win32/opengl/rgba-mode-and-windows-palette-management>
- [3] Hearn, D., & Baker, M. P. (2004). *Computer graphics with OpenGL* (3.^a ed.). Pearson Prentice Hall.
- [4] Marschner, S., & Shirley, P. (2021). *Fundamentals of computer graphics* (5.^a ed.). CRC Press.
- [5] Shreiner, D., Sellers, G., Kessenich, J., & Licea-Kane, B. (2013). *OpenGL programming guide: The official guide to learning OpenGL, version 4.3* (8.^a ed.). Addison-Wesley.
- [6] de Vries, J. (s.f.). *Hello triangle*. LearnOpenGL. Recuperado de <https://learnopengl.com/Getting-started/Hello-Triangle>
- [7] Khronos Group. (s.f.). *glPolygonMode - OpenGL 4 Reference Pages*. Recuperado de <https://registry.khronos.org/OpenGL-Refpages/gl4/html/glPolygonMode.xhtml>
- [8] G-Truc Creation. (s.f.). *OpenGL Mathematics (GLM) manual*. Recuperado de <https://glm.g-truc.net/0.9.9/index.html>
- [9] GLFW. (s.f.). *Window guide: Buffer swapping*. Recuperado de https://www.glfw.org/docs/latest/window_guide.html
- [10] cppreference.com. (s.f.). *Default initialization*. Recuperado de https://en.cppreference.com/w/cpp/language/default_initialization
- [11] Teodoro Vite, S. (s.f.). *Manual de prácticas. Práctica 03: Modelado geométrico* [Manual de laboratorio]. Facultad de Ingeniería, Universidad Nacional Autónoma de México.
- [12] Anthropic. (2026). *Claude* (Opus 5) [Modelo de lenguaje de gran escala]. <https://claude.ai> *Se utilizó Claude como apoyo para analizar el comportamiento de la clase Circle, en cambios de colores en figuras del proyecto base y en la revisión de redacción del presente reporte. El código base fue proporcionado por el profesor; las modificaciones al mismo, las capturas y los resultados son de autoría propia.*

Video Demostrativo

Enlace al funcionamiento de las tres actividades de la práctica:

<https://canva.link/6vtcze1bhsk34h6>